



Uniwersytet Rzeszowski
Wydział Nauk Ścisłych i Technicznych

PROGRAMOWANIE OBIEKTOWE II

*Dokumentacja projektowa - Wewnętrzny system
identyfikacji osób*

Filip Lisowski, Konrad Fugiel

Nr albumu: 134942, 134909

Kierunek studiów: Informatyka

Prowadzący przedmiotu: mgr inż. Ewa Żesławska

Rzeszów, 2026

Streszczenie

Praca przedstawia projekt i implementację aplikacji *IdentityCore*, będącej lokalnym systemem identyfikacji osób na podstawie danych biometrycznych. System został przygotowany jako aplikacja desktopowa w technologii WPF/.NET, współpracująca z lokalną bazą danych SQL Server. Głównym celem projektu było stworzenie prototypu umożliwiającego prowadzenie rejestru osób, zapisywanie danych biometrycznych, wykonywanie prób identyfikacji oraz analizę uzyskanych wyników.

W projekcie zastosowano dwa rodzaje biometrii: identyfikację twarzy oraz identyfikację odcisku palca. Moduł twarzy wykorzystuje model `arcface.onnx` do uzyskania reprezentacji cech twarzy, a następnie porównuje ją z zapisanymi wzorcami. Moduł odcisku palca wykorzystuje bibliotekę SourceAFIS do tworzenia i porównywania szablonów biometrycznych. Aplikacja uwzględnia również progi decyzyjne oraz margines pomiędzy najlepszym i drugim kandydatem, co pozwala odrzucać przypadki o zbyt niskim lub niejednoznacznym wyniku.

W ramach projektu przygotowano interfejs użytkownika obejmujący logowanie operatora, panel główny, rejestr osób, widoki identyfikacji twarzy i odcisku palca, historię prób oraz ustawienia systemowe. Przeprowadzone testy potwierdziły działanie podstawowych scenariuszy: poprawnego dopasowania oraz braku dopasowania dla obu typów biometrii. Wyniki mają charakter demonstracyjny i nie stanowią pełnej statystycznej walidacji systemu. Projekt pokazuje jednak kompletny przepływ działania prototypu systemu biometrycznego oraz wskazuje możliwe kierunki dalszego rozwoju, w tym zwiększenie bezpieczeństwa, rozszerzenie testów i dodanie mechanizmów wykrywania żywotności próbki.

Abstract

This project presents the design and implementation of *IdentityCore*, a local biometric identification system for persons. The system was developed as a desktop application using WPF/.NET and a local SQL Server database. The main objective of the project was to create a prototype that supports person registration, biometric data management, identification attempts, result presentation and identification history analysis.

The application uses two biometric modalities: face identification and fingerprint identification. The face module uses the `arcface.onnx` model to generate a numerical representation of facial features, which is then compared with previously stored templates. The fingerprint module uses the SourceAFIS library to create and compare biometric fingerprint templates. The system also applies decision thresholds and a margin between the best and the second-best candidate, allowing uncertain or low-quality matches to be rejected.

The project includes a user interface with operator login, dashboard, person registry, face identification view, fingerprint identification view, identification history and system settings. The performed tests confirmed the operation of the basic scenarios: successful matching and no-match decisions for both biometric modules. The results should be treated as a demonstration of a prototype rather than a full statistical validation of a biometric system. Nevertheless, the project demonstrates a complete workflow of a biometric identification application and indicates possible directions for further development, including improved security, extended testing and the introduction of liveness detection mechanisms.

Spis treści

1. Wprowadzenie	10
1.1. Kontekst projektu	10
1.2. Cel projektu	10
1.3. Zakres projektu	10
1.4. Charakter projektu łączonego	11
2. Analiza problemu i istniejących rozwiązań	12
2.1. Opis problemu	12
2.2. Dane biometryczne wykorzystywane w projekcie	12
2.3. Założenia i ograniczenia systemu	13
2.4. Analiza istniejących rozwiązań	13
2.4.1. Windows Hello	13
2.4.2. Face ID	14
2.4.3. Systemy kontroli dostępu z odciskiem palca	14
2.4.4. Komercyjne SDK i systemy biometryczne	14
2.5. Wyróżniki projektowanego rozwiązania	15
3. Wymagania systemowe	16
3.1. Wymagania funkcjonalne	16
3.2. Wymagania нефункционалне	17
3.3. Wymagania dotyczące bezpieczeństwa	18
3.4. Założenia dotyczące użytkowników i środowiska pracy	19
4. Koncepcja rozwiązania biometrycznego	20
4.1. Ogólna idea działania systemu	20
4.2. Pipeline przetwarzania danych	20
4.3. Proces rejestracji osoby	21
4.4. Proces identyfikacji osoby	21
4.5. Schemat blokowy działania systemu	22
5. Zastosowane metody i algorytmy	23
5.1. Przygotowanie danych wejściowych	23
5.2. Metoda identyfikacji twarzy	23
5.3. Metoda identyfikacji odcisku palca	24
5.4. Parametry decyzyjne systemu	24
5.5. Uzasadnienie wyboru metod	24
6. Architektura aplikacji	26
6.1. Ogólna architektura systemu	26
6.2. Podział na warstwy i moduły	26
6.3. Struktura projektu i organizacja kodu	28

6.4.	Zastosowane wzorce i zasady projektowe	29
6.5.	Walidacja danych	29
6.6.	Obsługa błędów i wyjątków	29
6.7.	Czytelność i utrzymanie projektu	29
7.	Technologie wykorzystane w projekcie	30
7.1.	Środowisko programistyczne	30
7.2.	Technologie interfejsu użytkownika	30
7.3.	Technologie dostępu do danych	30
7.4.	Biblioteki do przetwarzania obrazu i biometrii	31
7.5.	Narzędzia pomocnicze i organizacyjne	31
7.6.	Uzasadnienie wyboru technologii	32
8.	Projekt bazy danych i model danych	33
8.1.	Założenia modelu danych	33
8.2.	Opis głównych encji	33
8.3.	Relacje między tabelami	34
8.4.	Diagram ERD	35
8.5.	Integralność danych	36
8.6.	Skrypty bazy danych i migracje	36
8.7.	Uzasadnienie przyjętego modelu danych	36
9.	Implementacja aplikacji	37
9.1.	Najważniejsze moduły systemu	37
9.2.	Implementacja rejestracji osoby	38
9.3.	Implementacja procesu biometrycznego	39
9.3.1.	Implementacja identyfikacji twarzy	39
9.3.2.	Implementacja identyfikacji odcisku palca	43
9.3.3.	Zapis wyniku identyfikacji w historii	46
9.4.	Implementacja logowania i autoryzacji	47
9.5.	Operacje CRUD	48
9.6.	Nawigacja i przepływ danych w aplikacji	48
10.	Prezentacja warstwy użytkowej	50
10.1.	Ogólny układ interfejsu	50
10.2.	Ekran logowania	50
10.3.	Panel główny aplikacji	51
10.4.	Rejestr osób i profil osoby	52
10.5.	Widok identyfikacji twarzy	53
10.6.	Widok identyfikacji odcisku palca	54
10.7.	Historia prób identyfikacji	55
10.8.	Ustawienia systemu	56
10.9.	Komunikaty i informacje zwrotne dla operatora	56
11.	Wyniki, testy i analiza skuteczności	57

11.1. Opis przygotowanych testów	57
11.2. Dane testowe	57
11.3. Wyniki działania systemu	58
11.3.1. Wyniki identyfikacji twarzy	58
11.3.2. Wyniki identyfikacji odcisku palca	59
11.4. Analiza poprawnych i błędnych rozpoznań	61
11.5. Wpływ parametrów na działanie systemu	61
11.6. Wnioski z eksperymentów	61
12. Analiza bezpieczeństwa systemu	62
12.1. Potencjalne zagrożenia	62
12.2. Ryzyko fałszowania danych biometrycznych	62
12.3. Błędy identyfikacji	62
12.4. Ocena odporności systemu	62
12.5. Możliwe usprawnienia bezpieczeństwa	63
13. Harmonogram realizacji projektu	64
13.1. Etapy realizacji	64
13.2. Harmonogram prac	64
13.3. Podział pracy	65
14. Instrukcja uruchomienia projektu	66
14.1. Wymagania systemowe	66
14.2. Konfiguracja środowiska	66
14.3. Konfiguracja bazy danych	67
14.4. Uruchomienie aplikacji	67
14.5. Dane testowe i konta testowe	67
15. Podsumowanie i wnioski końcowe	69
15.1. Ocena realizacji założeń	69
15.2. Napotkane problemy	69
15.3. Ocena skuteczności rozwiązania	69
15.4. Możliwości zastosowania systemu	70
15.5. Kierunki dalszego rozwoju	70
Bibliografia	71
Spis rysunków	72
Spis tabel	73
Spis listingów	74
Oświadczenie studenta o samodzielności pracy	75

1. Wprowadzenie

1.1. Kontekst projektu

Systemy identyfikacji osób są wykorzystywane wszędzie tam, gdzie konieczne jest powiązanie danych zapisanych w systemie z konkretną osobą. W typowych rozwiązaniach informatycznych identyfikacja może opierać się na loginie, hasle, karcie dostępu lub numerze ewidencyjnym. Takie metody są proste w użyciu, ale nie zawsze potwierdzają fizyczną obecność właściwej osoby. Z tego powodu w systemach bezpieczeństwa coraz częściej wykorzystuje się dane biometryczne, czyli cechy bezpośrednio związane z człowiekiem.

Projekt *IdentityCore* dotyczy aplikacji służącej do wewnętrznej identyfikacji osób z wykorzystaniem dwóch cech biometrycznych: twarzy oraz odcisku palca. System został przygotowany jako aplikacja lokalna, której zadaniem jest obsługa rejestru osób, danych biometrycznych oraz prób identyfikacji. Przyjęty scenariusz odpowiada uproszczonemu systemowi wykorzystywanemu w środowisku demonstracyjnym lub laboratoryjnym, w którym operator może zarządzać profilami osób i sprawdzać wyniki identyfikacji.

Projekt nie jest gotowym systemem produkcyjnym przeznaczonym do pracy w środowisku wysokiego ryzyka, ale pozwala zaprezentować pełny przebieg tworzenia aplikacji łączącej część użytkową, bazodanową oraz biometryczną.

1.2. Cel projektu

Głównym celem projektu było zaprojektowanie i wykonanie aplikacji *IdentityCore*, która umożliwia identyfikację osób na podstawie danych biometrycznych. Aplikacja ma pozwalać na zapis danych osoby, powiązanie ich z próbkami biometrycznymi, wykonanie próby identyfikacji oraz zaprezentowanie wyniku w czytelnej formie.

Cel projektu obejmuje dwa uzupełniające się obszary. Pierwszy z nich dotyczy części biometrycznej, czyli wykorzystania obrazów twarzy i odcisków palców do rozpoznawania osób. Drugi dotyczy części aplikacyjnej, obejmującej zaprojektowanie kompletnego programu desktopowego z interfejsem użytkownika, bazą danych, obsługą logowania, historią działań oraz uporządkowaną strukturą kodu.

W efekcie projekt ma pokazać nie tylko samą ideę identyfikacji biometrycznej, ale również sposób jej osadzenia w działającej aplikacji użytkowej.

1.3. Zakres projektu

Zakres projektu obejmuje przygotowanie aplikacji umożliwiającej podstawową obsługę procesu wewnętrznej identyfikacji osób. W ramach systemu przewidziano następujące obszary funkcjonalne:

- dostęp do aplikacji poprzez ekran logowania,
- prowadzenie rejestru osób,
- obsługę danych opisowych i zdjęcia profilowego osoby,
- zapis i wykorzystanie danych biometrycznych twarzy,
- zapis i wykorzystanie danych biometrycznych odcisku palca,
- wykonanie próby identyfikacji osoby,

- prezentację wyniku identyfikacji,
- zapis historii wykonanych prób,
- konfigurację parametrów wpływających na decyzję systemu.

1.4. Charakter projektu łączonego

Dokumentacja opisuje jeden spójny projekt realizowany na potrzeby dwóch przedmiotów: *Biometryczne Systemy Zabezpieczeń* oraz *Programowanie Obiektowe II*. Z tego powodu aplikacja została opracowana tak, aby łączyć wymagania związane z analizą danych biometrycznych oraz wymagania dotyczące tworzenia aplikacji obiektowej.

Część związana z biometrią koncentruje się na wykorzystaniu danych twarzy i odcisku palca w procesie identyfikacji osoby. Obejmuje ona przygotowanie danych wejściowych, utworzenie reprezentacji biometrycznej, porównanie z danymi zapisanymi w systemie oraz interpretację wyniku.

Część związana z programowaniem obiektowym obejmuje zaprojektowanie aplikacji desktopowej, podział projektu na logiczne moduły, komunikację z bazą danych, obsługę interfejsu użytkownika, walidację danych oraz zapis historii działania systemu.

Takie połączenie pozwala traktować *IdentityCore* jako kompletną aplikację, w której mechanizmy biometryczne nie są oderwanym eksperymentem, lecz częścią większego systemu użytkowego.

2. Analiza problemu i istniejących rozwiązań

2.1. Opis problemu

Identyfikacja osoby w systemie informatycznym polega na ustaleniu, kto korzysta z systemu lub do kogo należy analizowany zestaw danych. W wielu aplikacjach proces ten opiera się na klasycznych metodach, takich jak login, hasło, karta dostępu, numer identyfikacyjny albo ręczna weryfikacja danych przez operatora. Rozwiązania te są powszechnie stosowane, ale w systemach bezpieczeństwa mogą być niewystarczające.

Hasło może zostać zapomniane, ujawnione lub przechwycone. Karta dostępową może zostać zgubiona albo przekazana innej osobie. Ręczna weryfikacja danych zależy natomiast od dokładności operatora i może być czasochłonna przy większej liczbie osób. W takich przypadkach sam identyfikator nie zawsze potwierdza, że dana osoba rzeczywiście jest tą, za którą się podaje.

Cechy biometryczne mogą wspierać ten proces, ponieważ są powiązane bezpośrednio z człowiekiem. Twarz i odcisk palca należą do najczęściej wykorzystywanych cech tego typu. Mogą zostać zarejestrowane w postaci obrazu, przetworzone do reprezentacji biometrycznej, a następnie porównane z danymi zapisanymi wcześniej w systemie. Dzięki temu identyfikacja nie opiera się wyłącznie na wiedzy użytkownika lub posiadanym nośniku, ale także na analizie cechy fizycznej.

W projekcie analizowany problem dotyczy identyfikacji osoby na podstawie wcześniej zarejestrowanych danych biometrycznych. System porównuje nową próbkę z danymi zapisanymi w bazie i wskazuje, czy odnaleziono dopasowanie.

2.2. Dane biometryczne wykorzystywane w projekcie

W projekcie wykorzystywane są dwa typy danych biometrycznych:

- obraz twarzy,
- obraz odcisku palca.

Obraz twarzy stanowi dane wejściowe pozwalające na utworzenie reprezentacji osoby na podstawie jej cech wyglądu. Tego typu dane mogą pochodzić z pliku graficznego lub z kamery. Dla poprawnego działania systemu ważne jest, aby twarz była widoczna, odpowiednio oświetlona i możliwa do przetworzenia.

Obraz odcisku palca jest drugim rodzajem danych wejściowych. W tym przypadku system wykorzystuje plik graficzny przedstawiający układ linii papilarnych. Obraz taki służy do utworzenia reprezentacji biometrycznej odcisku, która może zostać porównana z wcześniej zapisanymi danymi.

Oba typy danych nie są traktowane jako zwykłe obrazy porównywane bezpośrednio piksel po pikselu. Ich rola polega na dostarczeniu informacji potrzebnych do utworzenia reprezentacji biometrycznej, która następnie może zostać wykorzystana w procesie identyfikacji. Szczegóły sposobu tworzenia i porównywania tych reprezentacji opisano w dalszym rozdziale dotyczącym metod i algorytmów.

2.3. Założenia i ograniczenia systemu

Przyjęto, że system nie pełni roli produkcyjnego systemu bezpieczeństwa, lecz służy do zaprezentowania procesu identyfikacji osób na podstawie danych biometrycznych.

Najważniejsze założenia przyjęte w projekcie są następujące:

- system jest obsługiwany przez operatora,
- identyfikacja odbywa się na podstawie danych wcześniej zapisanych w bazie,
- analizowane dane mają postać obrazów twarzy i odcisków palców,
- skuteczność identyfikacji zależy od jakości danych wejściowych,
- wynik systemu ma wspierać proces identyfikacji, a nie zastępować pełnej ekspertyzy biometrycznej.

Na działanie systemu wpływa jakość obrazu. W przypadku twarzy znaczenie mają między innymi oświetlenie, ostrość zdjęcia, kąt ustawienia głowy oraz zasłonięcie części twarzy. W przypadku odcisku palca istotne są kompletność odcisku, jakość skanu, położenie palca oraz widoczność charakterystycznych elementów linii papilarnych.

Istotnym ograniczeniem jest również brak pełnej detekcji żywotności użytkownika. System analizuje dostarczony obraz, ale nie potwierdza, czy próbka pochodzi od żywej osoby obecnej w momencie identyfikacji.

Możliwym skutkiem ograniczeń jakościowych jest błędna identyfikacja albo odrzucenie poprawnej osoby. Szczegółowa analiza takich zagrożeń oraz odporności systemu została wydzielona do rozdziału poświęconego bezpieczeństwu.

2.4. Analiza istniejących rozwiązań

2.4.1. Windows Hello

Windows Hello jest mechanizmem uwierzytelniania użytkownika w systemie Windows. Może wykorzystywać między innymi rozpoznawanie twarzy, odcisk palca lub kod PIN [4]. Rozwiązanie to jest stosowane głównie do logowania do urządzenia oraz potwierdzania tożsamości użytkownika w środowisku systemu operacyjnego.

Zaletą Windows Hello jest wygoda użytkowania i integracja z systemem Windows. Użytkownik nie musi za każdym razem wpisywać hasła, a proces uwierzytelniania odbywa się szybko i lokalnie na urządzeniu. Ograniczeniem z perspektywy projektu jest zamknięty charakter rozwiązania. Użytkownik nie ma pełnego wglądu w sposób przetwarzania danych biometrycznych, strukturę zapisanych reprezentacji ani szczegóły podejmowania decyzji.

W odniesieniu do projektu IdentityCore Windows Hello stanowi przykład praktycznego wykorzystania biometrii w aplikacjach codziennego użytku. Różnica polega na tym, że IdentityCore nie jest mechanizmem logowania do systemu operacyjnego, lecz aplikacją demonstracyjną pozwalającą obserwować cały proces zarządzania osobami i próbami identyfikacji.

2.4.2. Face ID

Face ID jest systemem rozpoznawania twarzy stosowanym w urządzeniach firmy Apple [1]. Jego głównym zadaniem jest odblokowywanie urządzenia, autoryzacja operacji oraz potwierdzanie tożsamości użytkownika w ramach ekosystemu Apple. System wykorzystuje biometrię twarzy i jest zintegrowany bezpośrednio ze sprzętem oraz oprogramowaniem urządzenia.

Najważniejszą zaletą Face ID jest wygoda i wysoki poziom integracji z urządzeniem. Proces uwierzytelniania jest szybki, a użytkownik nie musi wykonywać dodatkowych czynności poza spojrzeniem na urządzenie. Ograniczeniem z punktu widzenia dokumentowanego projektu jest to, że Face ID działa jako zamknięty mechanizm systemowy. Nie jest rozwiązaniem służącym do prowadzenia własnego rejestru osób, analizy historii prób ani eksperymentowania z parametrami działania.

Dla projektu IdentityCore Face ID jest punktem odniesienia w zakresie wykorzystania twarzy jako cechy biometrycznej. Projekt nie próbuje odtwarzać tak zaawansowanego systemu sprzętowo-programowego, lecz pokazuje uproszczony proces identyfikacji twarzy w aplikacji desktopowej.

2.4.3. Systemy kontroli dostępu z odciskiem palca

Systemy kontroli dostępu wykorzystujące odciski palców są stosowane między innymi w firmach, laboratoriach, zamkach elektronicznych oraz systemach ewidencji czasu pracy. Ich zadaniem jest sprawdzenie, czy osoba przykładająca palec do czytnika znajduje się w bazie uprawnionych użytkowników [3].

Zaletą takich systemów jest prostota użycia oraz bezpośrednie powiązanie próby dostępu z cechą fizyczną użytkownika. Odcisk palca jest wygodny w codziennym użyciu i może być szybko porównany z zapisanymi wzorcami. Ograniczenia dotyczą przede wszystkim jakości odczytu, zabrudzenia palca lub skanera, nieprawidłowego przyłożenia palca oraz ryzyka użycia sztucznie przygotowanego odcisku.

W porównaniu z takimi rozwiązaniami IdentityCore nie jest terminalem kontroli dostępu. Aplikacja nie otwiera drzwi ani nie zarządza uprawnieniami fizycznymi. Wykorzystuje jednak podobny typ danych biometrycznych i pozwala przeanalizować proces identyfikacji odcisku w środowisku projektowym.

2.4.4. Komercyjne SDK i systemy biometryczne

Istnieją również komercyjne zestawy SDK oraz systemy biometryczne umożliwiające budowę aplikacji wykorzystujących rozpoznawanie twarzy, odcisków palców lub innych cech biometrycznych [5]. Tego typu rozwiązania oferują gotowe mechanizmy obsługi danych biometrycznych, porównywania próbek, a czasem również integrację z urządzeniami zewnętrznymi.

Ich zaletą jest szeroki zakres funkcji oraz możliwość użycia w profesjonalnych systemach bezpieczeństwa. Ograniczeniem w kontekście projektu jest często zamknięty charakter bibliotek oraz mniejsza przejrzystość działania. Część procesu rozpoznawania pozostaje ukryta wewnątrz gotowego narzędzia.

IdentityCore korzysta z gotowych narzędzi jedynie tam, gdzie jest to uzasadnione zakresem projektu, ale samodzielnie organizuje przepływ danych, rejestr osób, zapis prób i prezentację wyników. Dzięki temu aplikacja pozwala lepiej pokazać zależność między jakością danych, sposobem ich porównania i decyzją systemu.

2.5. Wyróżniki projektowanego rozwiązania

IdentityCore różni się od opisanych rozwiązań przede wszystkim tym, że łączy kilka elementów w jednej aplikacji desktopowej. System nie jest wyłącznie mechanizmem logowania ani terminalem kontroli dostępu, lecz aplikacją pozwalającą zarządzać osobami, próbkami biometrycznymi oraz wynikami prób identyfikacji.

Najważniejsze wyróżniki projektowanego rozwiązania to:

- połączenie identyfikacji twarzy i odcisku palca w jednej aplikacji,
- możliwość zarządzania osobami i próbkami biometrycznymi,
- zapis historii prób identyfikacji,
- możliwość późniejszej analizy skuteczności działania systemu.

Projekt nie konkuruje z komercyjnymi systemami biometrycznymi pod względem poziomu zabezpieczeń ani gotowości produkcyjnej. Jego wartość polega na pokazaniu kompletnego procesu identyfikacji w aplikacji, która łączy część biometryczną, użytkową i analityczną.

3. Wymagania systemowe

3.1. Wymagania funkcjonalne

Wymagania funkcjonalne określają funkcje, które system powinien udostępniać operatorowi. W przypadku aplikacji *IdentityCore* dotyczą one przede wszystkim obsługi rejestru osób, danych biometrycznych, procesu identyfikacji oraz historii wykonanych prób.

Id	Nazwa wymagania	Opis wymagania	Priorytet
WF-01	Logowanie operatora	System powinien umożliwiać operatorowi zalogowanie się do aplikacji przed uzyskaniem dostępu do głównych funkcji.	Wysoki
WF-02	Panel główny	System powinien udostępniać panel główny prezentujący podstawowe informacje o stanie danych i ostatnich działaniach.	Średni
WF-03	Rejestr osób	System powinien umożliwiać przeglądanie osób zapisanych w rejestrze.	Wysoki
WF-04	Dodawanie osoby	System powinien umożliwiać dodanie nowej osoby wraz z podstawowymi danymi opisowymi.	Wysoki
WF-05	Edycja danych osoby	System powinien umożliwiać aktualizację danych osoby zapisanej w rejestrze.	Wysoki
WF-06	Zdjęcie profilowe	System powinien umożliwiać przypisanie zdjęcia profilowego do osoby.	Średni
WF-07	Próbki twarzy	System powinien umożliwiać zapis danych biometrycznych twarzy przypisanych do osoby.	Wysoki
WF-08	Próbki odcisku palca	System powinien umożliwiać zapis danych biometrycznych odcisku palca przypisanych do osoby.	Wysoki
WF-09	Identyfikacja twarzy	System powinien umożliwiać wykonanie próby identyfikacji osoby na podstawie obrazu twarzy.	Wysoki
WF-10	Identyfikacja odcisku palca	System powinien umożliwiać wykonanie próby identyfikacji osoby na podstawie obrazu odcisku palca.	Wysoki
WF-11	Prezentacja wyniku	System powinien prezentować wynik identyfikacji w formie informacji o znalezionym dopasowaniu albo jego braku.	Wysoki
WF-12	Historia prób	System powinien umożliwiać zapis informacji o wykonanych próbach identyfikacji.	Wysoki

Id	Nazwa wymagania	Opis wymagania	Priorytet
WF-13	Przeglądanie historii	System powinien umożliwiać przeglądanie i filtrowanie historii prób identyfikacji.	Średni
WF-14	Ustawienia systemowe	System powinien umożliwiać konfigurację wybranych parametrów wpływających na decyzję identyfikacyjną.	Średni
WF-15	Komunikaty systemowe	System powinien informować operatora o podstawowych błędach, brakach danych lub wyniku wykonanej operacji.	Wysoki

Tabela 3.1. Wymagania funkcjonalne systemu

Wymagania funkcjonalne opisują zakres pracy systemu z perspektywy operatora. Nie określają sposobu realizacji poszczególnych funkcji, lecz wskazują, jakie działania powinny być możliwe w gotowej aplikacji.

3.2. Wymagania niefunkcjonalne

Wymagania niefunkcjonalne określają cechy jakościowe systemu. Dotyczą między innymi środowiska pracy, czytelności interfejsu, stabilności działania oraz organizacji projektu.

Id	Nazwa wymagania	Opis wymagania	Priorytet
WNF-01	Środowisko Windows	Aplikacja powinna działać w środowisku systemu Windows.	Wysoki
WNF-02	Praca lokalna	System powinien działać lokalnie, bez konieczności korzystania z zewnętrznego serwera.	Wysoki
WNF-03	Baza danych	Dane systemu powinny być przechowywane w lokalnej relacyjnej bazie danych.	Wysoki
WNF-04	Czytelność interfejsu	Interfejs użytkownika powinien być spójny, czytelny i dostosowany do obsługi.	Wysoki
WNF-05	Stabilność działania	System powinien obsługiwać typowe sytuacje błędne bez przerywania pracy aplikacji.	Wysoki
WNF-06	Obsługa błędów	Komunikaty błędów powinny być zrozumiałe i wskazywać przyczynę problemu.	Wysoki
WNF-07	Uporządkowanie danych	Dane osób, próbek biometrycznych, ustawień i historii prób powinny być przechowywane w uporządkowanej strukturze.	Wysoki
WNF-08	Przejrzystość kodu	Kod aplikacji powinien być podzielony zgodnie z odpowiedzialnością poszczególnych elementów systemu.	Średni

Tabela 3.2. Wymagania niefunkcjonalne systemu

Wymagania niefunkcjonalne są istotne, ponieważ projekt nie ogranicza się do samego działania funkcji. Aplikacja powinna być możliwa do uruchomienia, czytelna w obsłudze i odporna na podstawowe błędy użytkownika.

3.3. Wymagania dotyczące bezpieczeństwa

Wymagania dotyczące bezpieczeństwa określają podstawowe oczekiwania związane z kontrolą dostępu, przechowywaniem danych oraz ograniczaniem ryzyka błędnej interpretacji wyniku. Szczegółowa analiza zagrożeń została przedstawiona w osobnym rozdziale, dlatego w tym miejscu wskazano jedynie wymagania systemowe.

Id	Nazwa wymagania	Opis wymagania	Priorytet
WB-01	Kontrola dostępu	Dostęp do głównych funkcji aplikacji powinien być poprzedzony logowaniem operatora.	Wysoki
WB-02	Dane osób	System powinien przechowywać dane osób w bazie danych w sposób uporządkowany i powiązany z właściwym profilem.	Wysoki
WB-03	Dane biometryczne	Dane biometryczne powinny być przypisywane do odpowiedniej osoby zapisanej w rejestrze.	Wysoki
WB-04	Historia prób	System powinien zapisywać informacje o wykonanych próbach identyfikacji, aby możliwa była późniejsza analiza działania systemu.	Wysoki
WB-05	Parametry decyzyjne	System powinien umożliwiać kontrolę parametrów wpływających na decyzję identyfikacyjną.	Średni
WB-06	Ograniczenie błędnej identyfikacji	System powinien uwzględniać mechanizmy ograniczające ryzyko przypadkowego wskazania niewłaściwej osoby.	Wysoki
WB-07	Informowanie o wyniku	System powinien jednoznacznie informować operatora, czy dopasowanie zostało znalezione, czy nie.	Wysoki
WB-08	Błędne dane wejściowe	System powinien reagować na podstawowe problemy z danymi wejściowymi, takie jak brak pliku, nieczytelny obraz lub brak danych porównawczych.	Wysoki

Tabela 3.3. Wymagania dotyczące bezpieczeństwa

Wymagania bezpieczeństwa w tym rozdziale mają charakter podstawowy. Nie oznaczają, że system spełnia wymagania produkcyjnego systemu ochrony dostępu. Ich celem jest wskazanie, jakie mechanizmy powinny ograniczać błędne użycie aplikacji w warunkach projektowych.

3.4. Założenia dotyczące użytkowników i środowiska pracy

System został zaprojektowany z myślą o obsłudze przez operatora. Operator korzysta z aplikacji lokalnie, zarządza danymi osób, uruchamia próby identyfikacji i analizuje ich wyniki. Nie zakłada się rozbudowanego modelu wielu użytkowników, ról ani uprawnień.

Przyjęto następujące założenia dotyczące użytkowników i środowiska pracy:

- aplikacja jest obsługiwana przez jednego operatora,
- system działa lokalnie na komputerze z systemem Windows,
- dane osób muszą zostać wcześniej zapisane w bazie,
- identyfikacja odbywa się względem osób już zarejestrowanych w systemie,
- skuteczność identyfikacji zależy od jakości dostarczonych danych wejściowych,
- projekt jest przeznaczony wyłącznie do prezentacji i testów,
- system nie stanowi kompletnego produkcyjnego rozwiązania bezpieczeństwa.

Założenia te wyznaczają granice odpowiedzialności systemu. Aplikacja ma umożliwić przeprowadzenie i ocenę procesu identyfikacji biometrycznej w kontrolowanych warunkach, a nie zastępować profesjonalny system kontroli dostępu.

4. Koncepcja rozwiązania biometrycznego

4.1. Ogólna idea działania systemu

Koncepcja systemu *IdentityCore* opiera się na prowadzeniu lokalnego rejestru osób oraz powiązaniu każdej osoby z danymi biometrycznymi. Dane te mogą dotyczyć twarzy lub odcisku palca. Po zapisaniu osoby i jej próbek system może wykonać próbę identyfikacji na podstawie nowej próbki wejściowej.

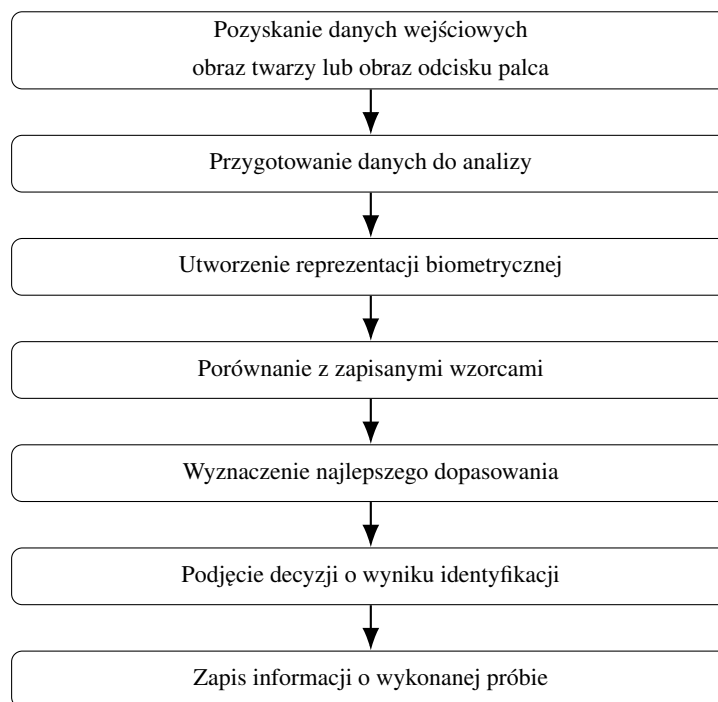
Identyfikacja odbywa się względem danych już dostępnych w systemie. Oznacza to, że analizowana próbka nie jest oceniana samodzielnie, lecz porównywana z wcześniej zapisanymi reprezentacjami biometrycznymi osób znajdujących się w rejestrze. Wynikiem procesu jest wskazanie najlepszego dopasowania albo informacja, że dopasowanie nie zostało znalezione.

4.2. Pipeline przetwarzania danych

Pipeline systemu biometrycznego opisuje ogólny przepływ danych od momentu dostarczenia próbki wejściowej do uzyskania wyniku identyfikacji. W przypadku obu wykorzystywanych modalności – twarzy oraz odcisku palca – proces ma podobną strukturę koncepcyjną: dane wejściowe są przygotowywane do analizy, przekształcane do postaci reprezentacji biometrycznej, a następnie porównywane z danymi zapisanymi wcześniej w systemie.

Na podstawie porównania system wyznacza najlepiej dopasowanego kandydata i podejmuje decyzję, czy wynik można uznać za poprawne dopasowanie. Informacja o wykonanej próbie może zostać zapisana w historii, co umożliwia późniejszą analizę oraz ocenę poprawności przeprowadzonych identyfikacji.

Na rysunku 4.1 przedstawiono uproszczony przepływ danych od momentu pozyskania próbki wejściowej do zapisania informacji o wykonanej próbie identyfikacji.



Rysunek 4.1. Ogólny pipeline działania systemu biometrycznego IdentityCore

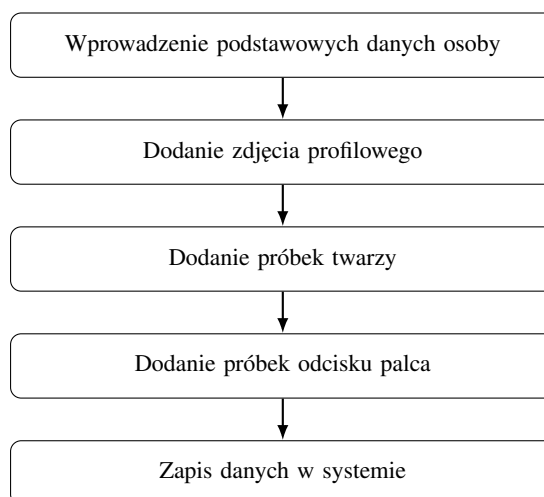
4.3. Proces rejestracji osoby

Proces rejestracji osoby polega na utworzeniu w systemie profilu, który może zostać później wykorzystany podczas identyfikacji. Nie jest to rejestracja konta użytkownika aplikacji, lecz dodanie wpisu osoby do rejestru identyfikacyjnego wraz z przypisanymi danymi opisowymi i biometrycznymi.

W praktyce proces ten obejmuje wprowadzenie podstawowych danych osoby, dodanie zdjęcia profilowego oraz zapisanie próbek biometrycznych twarzy i odcisku palca. Dane opisowe oraz zdjęcie profilowe służą do czytelnej prezentacji osoby w aplikacji, natomiast próbki biometryczne stanowią dane odniesienia wykorzystywane podczas późniejszych prób identyfikacji.

Osoba, dla której nie zapisano danych biometrycznych danego typu, nie może zostać skutecznie zidentyfikowana z użyciem tej modalności. Oznacza to, że jakość i kompletność danych zebranych na etapie rejestracji mają bezpośredni wpływ na późniejszą skuteczność działania systemu.

Na rysunku 4.2 przedstawiono uproszczony przebieg procesu rejestracji osoby w systemie *IdentityCore*.



Rysunek 4.2. Proces rejestracji osoby w systemie *IdentityCore*

4.4. Proces identyfikacji osoby

Proces identyfikacji rozpoczyna się od wyboru typu identyfikacji przez operatora. System może wykonać próbę na podstawie twarzy albo na podstawie odcisku palca. W obu przypadkach główna idea pozostaje taka sama: nowa próbka jest porównywana z danymi zapisanymi wcześniej w systemie.

W przypadku identyfikacji twarzy operator dostarcza obraz twarzy. System przygotowuje dane wejściowe do analizy, tworzy ich reprezentację biometryczną i porównuje ją z reprezentacjami zapisanymi dla osób w rejestrze. Jeżeli wynik porównania spełnia przyjęte warunki, system wskazuje najlepiej dopasowaną osobę. W przeciwnym razie informuje o braku dopasowania.

W przypadku identyfikacji odcisku palca operator dostarcza obraz odcisku. System przetwarza próbkę do postaci umożliwiającej porównanie z zapisanymi danymi biometrycznymi. Następnie wybierany jest najlepszy kandydat albo zwracana jest informacja, że nie znaleziono osoby odpowiadającej analizowanej próbce.

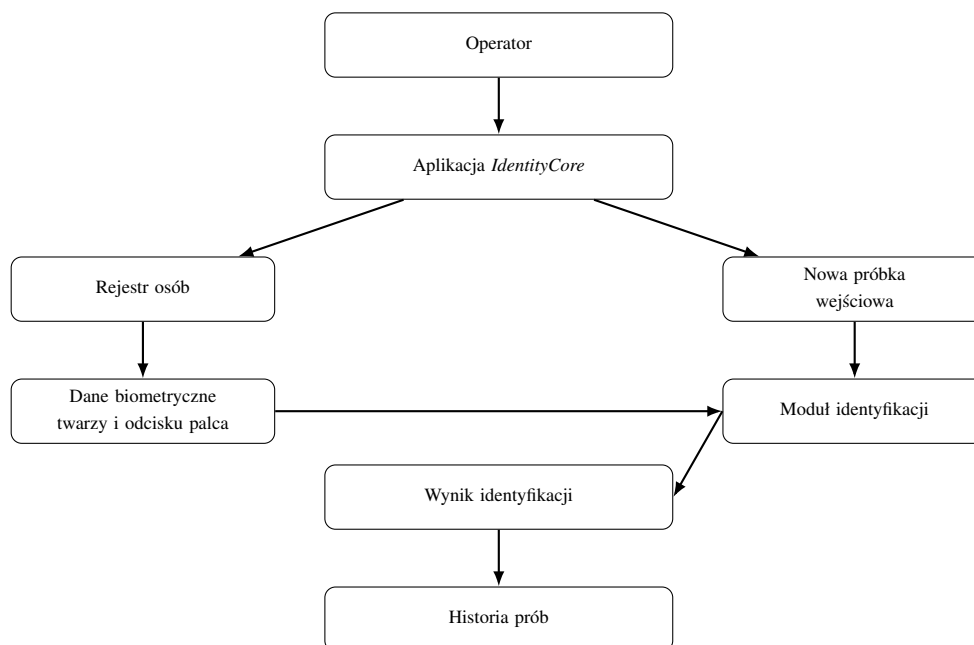
Po zakończeniu próby wynik może zostać zapisany w historii. Zapis obejmuje podstawowe informacje o wykonanej identyfikacji, takie jak typ użytej biometrii, wynik oraz powiązanie z rozpoznaną osobą, jeżeli dopasowanie zostało znalezione.

4.5. Schemat blokowy działania systemu

Schemat blokowy systemu przedstawia ogólną organizację przepływu danych i decyzji w aplikacji *IdentityCore*. W odróżnieniu od pipeline'u przetwarzania danych, schemat ten pokazuje nie tylko kolejne etapy analizy próbki, ale również powiązanie operatora, rejestru osób, danych biometrycznych, modułu identyfikacji, wyniku oraz historii prób.

Na poziomie koncepcyjnym można wyróżnić dwa główne tory działania systemu. Pierwszy dotyczy przygotowania danych odniesienia, czyli zapisania osoby w rejestrze oraz powiązania jej z danymi biometrycznymi. Drugi dotyczy wykonania próby identyfikacji, w której nowa próbka wejściowa jest porównywana z danymi zapisanymi wcześniej w systemie. Wynik próby może zostać następnie zapisany w historii.

Na rysunku 4.3 przedstawiono uproszczony schemat blokowy działania systemu. Diagram ma charakter koncepcyjny i nie przedstawia szczegółów implementacyjnych.



Rysunek 4.3. Schemat blokowy działania systemu *IdentityCore*

5. Zastosowane metody i algorytmy

5.1. Przygotowanie danych wejściowych

W systemie *IdentityCore* dane wejściowe mają postać obrazów. W zależności od typu identyfikacji analizowany jest obraz twarzy albo obraz odcisku palca. Oba typy danych muszą zostać przygotowane tak, aby możliwe było utworzenie reprezentacji biometrycznej porównywanej później z danymi zapisanymi w systemie.

W przypadku twarzy duże znaczenie ma jakość kadru. Na skuteczność identyfikacji wpływają przede wszystkim oświetlenie, ostrość, kąt ustawienia głowy, widoczność twarzy oraz elementy zasłaniające. Obraz powinien zawierać możliwie wyraźny obszar twarzy, ponieważ to on stanowi podstawę dalszej analizy.

W przypadku odcisku palca jakość danych zależy od czytelności linii papilarnych. Znaczenie mają między innymi kompletność odcisku, kontrast, zabrudzenia, rozmazanie oraz sposób przyłożenia palca. Niepełny lub słabo widoczny odcisk może utrudnić utworzenie poprawnego szablonu i obniżyć skuteczność identyfikacji.

Przygotowanie danych wejściowych jest więc istotnym etapem procesu biometrycznego. Jakość próbki wpływa bezpośrednio na wynik porównania, dlatego dane powinny być pozyskiwane w możliwie powtarzalnych warunkach.

5.2. Metoda identyfikacji twarzy

Identyfikacja twarzy w projekcie opiera się na porównywaniu numerycznych reprezentacji cech twarzy. Dane wejściowe stanowi obraz zawierający twarz osoby. Przed porównaniem obraz jest przygotowywany tak, aby dalsza analiza dotyczyła głównie obszaru twarzy, a nie całego kadru. Ogranicza to wpływ tła i elementów niezwiązanych z identyfikowaną osobą.

Do utworzenia reprezentacji twarzy wykorzystano model ArcFace zapisany w formacie ONNX [2, 6]. Model przekształca obraz twarzy do postaci wektora cech, określanego jako embedding. Nie jest on obrazem, lecz numerycznym opisem cech twarzy, który można porównać z embeddingami zapisanymi wcześniej w systemie.

W projekcie podobieństwo pomiędzy embeddingiem próbki wejściowej a embeddingami zapisanymi w systemie jest obliczane z użyciem podobieństwa cosinusowego. Miara ta określa zgodność kierunku dwóch wektorów cech:

$$\cos(\theta) = \frac{A \cdot B}{\|A\| \|B\|}$$

gdzie A i B oznaczają porównywane wektory. Im większa wartość podobieństwa, tym bardziej zbliżone są reprezentacje porównywanych twarzy.

Na podstawie otrzymanych wartości system wybiera najlepszego kandydata spośród osób zapisanych w rejestrze. Decyzja o rozpoznaniu zależy jednak także od progu decyzyjnego oraz marginesu pomiędzy najlepszym i drugim wynikiem. Ogranicza to ryzyko wskazania osoby, gdy wynik jest zbyt niski albo różnica między kandydatami jest niewystarczająca.

Implementacyjny fragment kodu obliczający podobieństwo cosinusowe został przedstawiony w rozdziale 9, w sekcji dotyczącej najważniejszych fragmentów kodu. W tym miejscu istotne jest wyjaśnienie roli tej miary, a nie omawianie szczegółów konkretnej klasy.

5.3. Metoda identyfikacji odcisku palca

Identyfikacja odcisku palca opiera się na analizie obrazu przedstawiającego układ linii papilarnych. Dane wejściowe stanowi plik graficzny zawierający odcisk palca. Obraz jest przygotowywany do analizy, a następnie przekształcany do postaci szablonu biometrycznego.

W projekcie do obsługi odcisków palców wykorzystano bibliotekę SourceAFIS [7]. Umożliwia ona utworzenie szablonu odcisku oraz porównanie go z innymi zapisanymi szablonami. Szablon biometryczny opisuje charakterystyczne cechy odcisku, związane między innymi z układem linii papilarnych i punktami charakterystycznymi.

Podczas identyfikacji szablon utworzony dla próbki wejściowej jest porównywany z szablonami zapisanymi wcześniej w systemie. Wynikiem porównania jest wartość dopasowania, która określa podobieństwo próbki wejściowej do danego wzorca. Na tej podstawie wybierany jest najlepszy kandydat.

W aktualnej konfiguracji projektu dla identyfikacji odcisku palca przyjęto próg dopasowania równy 85 oraz margines bezpieczeństwa równy 15 punktów. Oznacza to, że wynik odcisku palca jest oceniany nie tylko na podstawie najlepszego dopasowania, ale również z uwzględnieniem różnicy względem kolejnego kandydata. Ogólną rolę tych parametrów opisano w następnej sekcji.

5.4. Parametry decyzyjne systemu

Parametry decyzyjne określają warunki, które muszą zostać spełnione, aby system uznał próbę identyfikacji za zakończoną znalezieniem dopasowania. Najważniejszym z nich jest próg akceptacji: dla twarzy dotyczy podobieństwa między embeddingami, a dla odcisku palca wartości dopasowania szablonów.

Drugim istotnym parametrem jest margines pomiędzy najlepszym i drugim najlepszym kandydatem. Ogranicza on sytuacje, w których system wskazuje osobę mimo niewielkiej różnicy między kilkoma wynikami.

W projekcie decyzja identyfikacyjna nie jest podejmowana wyłącznie na podstawie najwyższego wyniku. System sprawdza również, czy najlepszy kandydat ma wystarczającą przewagę nad kolejnym, dzięki czemu decyzja jest ostrożniejsza.

Przykład implementacji decyzji opartej na progu dopasowania i marginesie został przedstawiony w rozdziale 9, w sekcji dotyczącej najważniejszych fragmentów kodu. W tym miejscu istotne jest wyjaśnienie, dlaczego takie parametry są potrzebne w procesie identyfikacji.

Dobór parametrów wpływa na działanie systemu. Zbyt niski próg może zwiększać ryzyko fałszywego dopasowania, natomiast zbyt wysoki może prowadzić do odrzucania poprawnych prób. Podobnie większy margines zwiększa ostrożność decyzji, ale może powodować więcej przypadków bez rozpoznania.

W projekcie parametry decyzyjne regulują kompromis między skutecznością rozpoznawania a ostrożnością decyzji. Szczegółowa ocena ich wpływu na wyniki została przewidziana w rozdziale poświęconym testom i analizie skuteczności.

5.5. Uzasadnienie wyboru metod

Wybór modelu ArcFace w formacie ONNX dla identyfikacji twarzy wynikał z potrzeby utworzenia reprezentacji cech twarzy w postaci embeddingu. Dzięki temu system nie porównuje obrazów bezpośrednio, lecz wykorzystuje numeryczne reprezentacje, które lepiej nadają się do oceny podobieństwa pomiędzy twarzami.

W przypadku odcisków palców zastosowanie biblioteki SourceAFIS pozwoliło oprzeć identyfikację na szablonach biometrycznych, a nie na prostym porównywaniu obrazów. Jest to podejście właściwsze dla

tego typu danych, ponieważ uwzględnia charakterystyczne cechy odcisku, a nie wyłącznie wygląd pikseli w obrazie.

W obu modalnościach przyjęto podobną logikę decyzyjną: próbka wejściowa jest przekształcana do reprezentacji biometrycznej, porównywana z zapisanymi wzorcami, a wynik oceniany przy użyciu progu i marginesu. Takie rozwiązanie pozwala pokazać pełny proces identyfikacji: od próbki wejściowej, przez reprezentację cech, aż po decyzję systemu zapisaną do dalszej oceny.

6. Architektura aplikacji

6.1. Ogólna architektura systemu

Aplikacja *IdentityCore* została zaprojektowana jako lokalna aplikacja desktopowa, w której wydzielono kilka głównych obszarów odpowiedzialności: warstwę prezentacji, logikę aplikacyjną, moduły biometryczne oraz warstwę dostępu do danych. Taki podział pozwala oddzielić elementy odpowiedzialne za wygląd aplikacji od części realizujących operacje identyfikacyjne i zapis danych.

Na poziomie architektury system można traktować jako aplikację zorganizowaną warstwowo. Widoki odpowiadają za prezentację danych, modele widoków za obsługę stanu ekranów i akcji operatora, serwisy za logikę aplikacyjną, a katalog *Data* za komunikację z bazą danych. Moduły biometryczne są częścią logiki aplikacyjnej, ale zostały wydzielone tak, aby można było rozwijać je niezależnie od warstwy interfejsu.

6.2. Podział na warstwy i moduły

Najważniejsze warstwy oraz moduły aplikacji przedstawiono w tabeli 6.1. Zestawienie pokazuje, za co odpowiadają poszczególne części projektu i gdzie znajdują się ich przykładowe elementy.

Moduł/warstwa	Odpowiedzialność	Przykładowe elementy projektu
Warstwa widoków	Prezentacja interfejsu użytkownika oraz obsługa ekranów aplikacji.	Katalog <i>Views</i> , między innymi widoki panelu głównego, rejestru osób, identyfikacji, historii, ustawień i logowania.
Modele widoków	Przechowywanie stanu ekranów, obsługa komend oraz pośredniczenie pomiędzy widokiem a logiką aplikacji.	Katalog <i>ViewModels</i> , między innymi modele widoków panelu głównego, rejestru osób, identyfikacji, historii i ustawień.
Serwisy aplikacyjne	Realizacja operacji wymagających osobnej logiki aplikacyjnej lub przetwarzania danych.	Katalog <i>Services</i> , między innymi serwisy logowania, kamery, przetwarzania obrazów, identyfikacji i ustawień.
Warstwa dostępu do danych	Odczyt, zapis i aktualizacja danych przechowywanych przez aplikację.	Katalog <i>Data</i> , między innymi repozytoria osób, szablonów biometrycznych, historii prób, panelu głównego oraz połączenia z bazą.
Modele danych	Reprezentacja obiektów wykorzystywanych w systemie oraz wyników operacji.	Katalog <i>Models</i> , między innymi modele osoby, próbek biometrycznych, wyników identyfikacji, kandydatów dopasowania i ustawień decyzyjnych.

Moduł/warstwa	Odpowiedzialność	Przykładowe elementy projektu
Moduł identyfikacji twarzy	Obsługa procesu związanego z analizą i porównywaniem danych twarzy.	Elementy w katalogach <code>Services</code> , <code>Models</code> , <code>Data</code> , <code>Assets</code> oraz <code>MLModels</code> .
Moduł identyfikacji odcisku palca	Obsługa procesu związanego z analizą i porównywaniem danych odcisku palca.	Elementy w katalogach <code>Services</code> , <code>Models</code> oraz <code>Data</code> .
Moduł historii prób	Przechowywanie i udostępnianie informacji o wykonanych próbach identyfikacji.	Widok historii, model widoku historii, repozytorium identyfikacji oraz model próby identyfikacji.
Moduł ustawień systemowych	Obsługa parametrów wpływających na działanie procesu identyfikacji.	Widok ustawień, model widoku ustawień, <code>SystemSettingsService</code> oraz model ustawień biometrycznych.

Tabela 6.1. Główne warstwy i moduły aplikacji

Podział przedstawiony w tabeli porządkuje odpowiedzialności w projekcie. Najważniejsze jest to, że widoki nie wykonują samodzielnie operacji biometrycznych ani nie obsługują bezpośrednio zapisu danych. Zadania te są przekazywane do odpowiednich modeli widoków, serwisów lub klas z warstwy `Data`.

6.3. Struktura projektu i organizacja kodu

Struktura katalogów pokazuje fizyczne rozmieszczenie poszczególnych elementów aplikacji w projekcie. Uproszczony układ katalogów przedstawiono poniżej.

```
IdentityCore.App/  
|-- Assets/  
|   '-- Cascades/  
|-- Data/  
|-- Database/  
|-- MLModels/  
|-- Models/  
|-- Services/  
|-- Styles/  
|-- ViewModels/  
|-- Views/  
|-- App.xaml  
|-- App.xaml.cs  
|-- AssemblyInfo.cs  
|-- IdentityCore.App.csproj  
|-- MainWindow.xaml  
'-- MainWindow.xaml.cs
```

Katalog `Assets` przechowuje katalog `Cascades` zawierający pliki kaskad wykorzystywane pomocniczo podczas pracy z obrazami twarzy.

Katalog `Data` zawiera klasy odpowiedzialne za dostęp do danych. W tym miejscu umieszczono połączenie z bazą danych oraz repozytoria obsługujące dane osób, próbek biometrycznych, szablonów twarzy, szablonów odcisków palców, historii identyfikacji, panelu głównego i logowania.

Katalog `Database` zawiera skrypty SQL służące do tworzenia, aktualizacji oraz czyszczenia danych biometrycznych w bazie. Szczegółowy opis struktury danych zostanie przedstawiony w osobnym rozdziale dotyczącym bazy danych.

Katalog `MLModels` przechowuje plik modelu wykorzystywany przez moduł identyfikacji twarzy. Oddzielenie tego typu zasobów od kodu aplikacji ułatwia utrzymanie porządku w projekcie.

Katalog `Models` zawiera klasy opisujące dane wykorzystywane w systemie, między innymi osoby, próbki biometryczne, wyniki identyfikacji, kandydatów dopasowania, szablony oraz ustawienia decyzyjne.

Katalog `Services` grupuje logikę aplikacyjną, która nie należy bezpośrednio do widoków. Obejmuje to między innymi obsługę logowania, kamery, przetwarzania obrazów, identyfikacji, diagnostyki, kalibracji i ustawień.

Katalog `Styles` zawiera współdzielone pliki XAML odpowiedzialne za wygląd elementów interfejsu, takich jak przyciski, karty, kolory, tabele i pola wejściowe.

Katalogi `ViewModels` i `Views` odpowiadają za warstwę użytkową aplikacji. Pierwszy z nich przechowuje modele widoków obsługujące stan ekranów i komendy, a drugi zawiera pliki widoków oraz powiązane z nimi pliki code-behind. Pliki `App.xaml` i `MainWindow.xaml` odpowiadają za uruchomienie aplikacji oraz główne okno programu.

6.4. Zastosowane wzorce i zasady projektowe

Organizacja aplikacji opiera się na podejściu MVVM. W projekcie oznacza to, że widok odpowiada za prezentację danych, a model widoku obsługuje stan ekranu, komendy i przekazywanie zadań do dalszych warstw aplikacji. Dzięki temu logika działania nie musi być umieszczana bezpośrednio w plikach widoków.

Serwisy odpowiadają za operacje wymagające osobnej logiki aplikacyjnej. Dotyczy to między innymi przetwarzania obrazów, obsługi kamery, logowania, ustawień oraz identyfikacji biometrycznej. Wydzielenie serwisów ogranicza rozrastanie się modeli widoków i pozwala utrzymać bardziej czytelny podział zadań.

Najważniejszą zasadą projektową jest separacja odpowiedzialności. Każdy element aplikacji ma określoną rolę: widok prezentuje dane, model widoku obsługuje zachowanie ekranu, serwis wykonuje logikę aplikacyjną, model opisuje dane, a warstwa `Data` odpowiada za ich trwały zapis i odczyt.

6.5. Walidacja danych

Walidacja danych dotyczy sprawdzenia poprawności informacji przed wykonaniem operacji. Jej celem jest ograniczenie sytuacji, w których aplikacja próbuje zapisać niepełne dane osoby, uruchomić identyfikację bez próbki wejściowej albo wykonać porównanie bez danych odniesienia.

W rejestrze osób walidacja obejmuje kontrolę wymaganych pól i podstawowej poprawności danych profilu. W modułach biometrycznych sprawdzane jest przede wszystkim to, czy dostępny jest plik lub obraz możliwy do dalszej analizy. Jeżeli dane są niekompletne, operacja nie powinna być wykonywana do momentu ich poprawienia.

Walidacja pełni więc funkcję zabezpieczenia przed typowymi błędami użytkownika aplikacji. Szczegółowy sposób prezentowania komunikatów operatorowi zostanie opisany w rozdziale dotyczącym warstwy użytkowej.

6.6. Obsługa błędów i wyjątków

Obsługa błędów dotyczy sytuacji, w których mimo sprawdzenia danych operacja nie może zostać wykonana poprawnie. Mogą to być problemy z połączeniem z bazą danych, niepowodzenie zapisu lub odczytu, niepoprawny plik wejściowy, błąd przetwarzania obrazu albo nieudana próba logowania.

W takich przypadkach aplikacja powinna reagować w sposób kontrolowany. Typowy problem nie powinien powodować nagłego przerwania działania programu, lecz zakończyć się przekazaniem operatorowi informacji o przyczynie niepowodzenia. Pozwala to poprawić dane, wybrać inny plik, ponowić próbę albo sprawdzić konfigurację środowiska.

Obsługa wyjątków uzupełnia walidację. Walidacja zapobiega części błędów przed rozpoczęciem operacji, natomiast obsługa wyjątków odpowiada za przypadki, których nie dało się przewidzieć lub które wystąpiły w trakcie działania aplikacji.

6.7. Czytelność i utrzymanie projektu

Przyjęty podział projektu ułatwia analizę i dalszą rozbudowę aplikacji. Najważniejsze znaczenie ma oddzielenie widoków od logiki aplikacyjnej, wydzielenie serwisów oraz skupienie dostępu do danych w katalogu `Data`. Dzięki temu projekt pozostaje czytelny i możliwy do rozwijania w ramach kolejnych etapów pracy.

7. Technologie wykorzystane w projekcie

7.1. Środowisko programistyczne

Aplikacja *IdentityCore* została przygotowana w języku C# z wykorzystaniem platformy .NET. Technologie te wykorzystano do utworzenia lokalnej aplikacji desktopowej realizującej logikę systemu, obsługę interfejsu użytkownika, komunikację z bazą danych oraz integrację z modułami biometrycznymi.

Projekt był rozwijany w środowisku Visual Studio 2026, wersja 18.5.2. Środowisko to umożliwiło edycję plików C# i XAML, zarządzanie zależnościami, uruchamianie aplikacji oraz debugowanie programu podczas prac projektowych.

Jako środowisko uruchomieniowe przyjęto system Windows. Jest to zgodne z charakterem aplikacji, ponieważ *IdentityCore* działa lokalnie. W pliku projektu wskazano platformę docelową `net10.0-windows`, co oznacza wykorzystanie platformy .NET 10 dla aplikacji Windows Desktop.

7.2. Technologie interfejsu użytkownika

Do wykonania interfejsu użytkownika zastosowano WPF oraz XAML. WPF został wykorzystany jako technologia aplikacji desktopowej, natomiast XAML posłużył do opisu struktury widoków i elementów wizualnych.

Interfejs aplikacji obejmuje ekrany odpowiadające głównym obszarom systemu, takim jak panel główny, rejestr osób, identyfikacja twarzy, identyfikacja odcisku palca, historia prób oraz ustawienia. W tym rozdziale istotne jest jednak samo wskazanie technologii, a nie szczegółowe omawianie scenariuszy użycia poszczególnych widoków.

Wspólne style interfejsu zapisano w plikach XAML. Pozwala to zachować spójność wyglądu przycisków, kart, tabel, pól wejściowych i kolorów w różnych częściach aplikacji. Takie rozwiązanie ułatwia utrzymanie jednolitego charakteru graficznego systemu bez powielania tych samych ustawień w każdym widoku.

7.3. Technologie dostępu do danych

Do przechowywania danych wykorzystano SQL Server jako relacyjną bazę danych. W projekcie baza służy do zapisu informacji o osobach, danych biometrycznych, ustawieniach systemowych oraz historii wykonanych prób identyfikacji. Szczegółowa struktura tabel i relacji zostanie przedstawiona w osobnym rozdziale dotyczącym bazy danych.

Komunikacja aplikacji z bazą danych jest realizowana z użyciem biblioteki `Microsoft.Data.SqlClient`. Biblioteka ta umożliwia wykonywanie operacji zapisu, odczytu i aktualizacji danych z poziomu aplikacji C#. Dzięki temu aplikacja może korzystać z danych zapisanych w relacyjnej bazie bez konieczności stosowania zewnętrznego serwera aplikacyjnego.

W projekcie wykorzystano również skrypty SQL. Służą one do przygotowania struktury bazy danych, aktualizacji istniejącej bazy oraz czyszczenia danych biometrycznych podczas prac testowych. Do pracy z bazą wykorzystano SQL Server Management Studio, które umożliwiło sprawdzanie struktury danych i kontrolę poprawności skryptów.

7.4. Biblioteki do przetwarzania obrazu i biometrii

Część biometryczna projektu korzysta z narzędzi wspierających pracę z obrazami oraz reprezentacjami biometrycznymi. Każde z nich pełni konkretną rolę w systemie i zostało dobrane do określonego obszaru aplikacji.

OpenCvSharp wykorzystano jako bibliotekę wspierającą przetwarzanie obrazów. W projekcie pomaga ona w operacjach związanych z obrazami twarzy, przygotowaniem danych wejściowych oraz obsługą materiału obrazowego. Nie jest to samodzielny mechanizm identyfikacji osoby, lecz narzędzie pomocnicze w pracy z obrazem.

Do obsługi identyfikacji twarzy wykorzystano model ArcFace zapisany w pliku `arcface.onnx`. Plik ten zawiera gotowy model służący do uzyskania reprezentacji twarzy. Format ONNX pełni w projekcie rolę sposobu zapisu modelu, natomiast jego uruchomienie w aplikacji jest realizowane z wykorzystaniem pakietów `Microsoft.ML.OnnxTransformer` oraz `Microsoft.ML.OnnxRuntime`. Dzięki temu model może zostać użyty bezpośrednio w aplikacji .NET.

Do obsługi odcisków palców zastosowano bibliotekę SourceAFIS. W projekcie odpowiada ona za tworzenie i porównywanie szablonów odcisków palców. Dzięki temu część związana z odciskiem palca może być realizowana na poziomie reprezentacji biometrycznej, a nie przez proste porównywanie obrazów.

W tym rozdziale biblioteki są omawiane z perspektywy technologii użytych w projekcie. Rozdział dotyczący metod i algorytmów przedstawiał zasadę działania identyfikacji twarzy i odcisku palca, natomiast szczegóły implementacyjne, klasy oraz szersze fragmenty kodu zostaną omówione w rozdziale poświęconym implementacji aplikacji. W tym miejscu istotne jest więc przede wszystkim wskazanie, jakie narzędzia zostały użyte i jaką pełnią rolę w systemie.

7.5. Narzędzia pomocnicze i organizacyjne

Podczas pracy nad projektem wykorzystano narzędzia wspierające organizację kodu, obsługę dużych plików oraz przygotowanie środowiska danych. Do kontroli wersji użyto systemu Git, a repozytorium projektu przechowywano w serwisie GitHub. Pozwoliło to śledzić zmiany w kodzie i porządkować kolejne etapy pracy.

W projekcie wykorzystano również pakiety NuGet. Służą one do zarządzania zależnościami aplikacji, między innymi bibliotekami związanymi z dostępem do danych, obsługą obrazów, uruchamianiem modelu ONNX oraz identyfikacją odcisków palców. Najważniejsze pakiety wykorzystane w projekcie przedstawiono w tabeli 7.1.

Pakiet NuGet	Wersja
<code>Microsoft.Data.SqlClient</code>	7.0.1
<code>Microsoft.ML</code>	5.0.0
<code>Microsoft.ML.OnnxRuntime</code>	1.26.0
<code>Microsoft.ML.OnnxTransformer</code>	5.0.0
<code>OpenCvSharp4</code>	4.13.0.20260427
<code>OpenCvSharp4.runtime.win</code>	4.13.0.20260302
<code>OpenCvSharp4.WpfExtensions</code>	4.13.0.20260427
<code>SourceAFIS</code>	3.14.0

Tabela 7.1. Najważniejsze pakiety NuGet wykorzystane w projekcie

Dodatkową rolę pełnią pliki graficzne i zasoby aplikacji, takie jak ikony, obrazy, style XAML oraz pliki pomocnicze używane przy pracy z obrazami. Nie są one osobnymi technologiami programistycznymi, ale wpływają na kompletność i spójność działania aplikacji.

7.6. Uzasadnienie wyboru technologii

Dobór technologii wynikał z charakteru projektu. Aplikacja miała działać lokalnie, posiadać interfejs desktopowy, korzystać z relacyjnej bazy danych oraz integrować moduły biometryczne. Z tego powodu C#/.NET i WPF/XAML dobrze pasują do przygotowania aplikacji obsługiwanej przez operatora w środowisku Windows.

SQL Server został wykorzystany do uporządkowanego przechowywania danych systemu, natomiast biblioteki biometryczne umożliwiły realizację identyfikacji twarzy i odcisku palca. Całość jest spójna z charakterem projektu łączącego zagadnienia z Programowania Obiektowego II oraz Biometrycznych Systemów Zabezpieczeń.

Technologia/narzędzie	Zastosowanie w projekcie	Uzasadnienie wyboru
C# i .NET	Logika aplikacji desktopowej oraz integracja poszczególnych części systemu.	Technologie odpowiednie do tworzenia lokalnej aplikacji dla systemu Windows.
WPF i XAML	Przygotowanie interfejsu użytkownika oraz wspólnych stylów aplikacji.	Umożliwiają budowę aplikacji desktopowej z czytelnym i spójnym interfejsem.
SQL Server	Przechowywanie danych osób, ustawień, danych biometrycznych i historii prób.	Relacyjna baza danych pozwala uporządkować dane wykorzystywane przez system.
Microsoft.Data.SqlClient	Komunikacja aplikacji z bazą danych.	Pozwala wykonywać operacje na SQL Server z poziomu aplikacji C#.
OpenCvSharp	Wsparcie operacji związanych z przetwarzaniem obrazów.	Ułatwia przygotowanie danych obrazowych wykorzystywanych w części biometrycznej.
ArcFace/ONNX + ONNX Runtime	Uzyskanie reprezentacji twarzy na podstawie pliku modelu <code>arcface.onnx</code> .	Pozwalają wykorzystać gotowy model twarzy w aplikacji .NET i połączyć go z logiką identyfikacji.
SourceAFIS	Obsługa szablonów i porównywania odcisków palców.	Biblioteka pasuje do części projektu odpowiedzialnej za identyfikację odcisku palca.
Git, GitHub i Git LFS	Kontrola wersji, przechowywanie repozytorium oraz obsługa dużych plików.	Ułatwiają organizację pracy i przechowywanie zasobów takich jak model ONNX.
SSMS	Przygotowanie i kontrola bazy danych.	Ułatwia pracę ze skryptami SQL oraz weryfikację danych.

Tabela 7.2. Podsumowanie technologii wykorzystanych w projekcie

8. Projekt bazy danych i model danych

8.1. Założenia modelu danych

Baza danych systemu *IdentityCore* została zaprojektowana jako relacyjny model danych przechowujący informacje potrzebne do działania lokalnego systemu identyfikacji osób. Jej zadaniem jest uporządkowane zapisanie danych osób, danych biometrycznych, historii prób identyfikacji, ustawień systemowych oraz danych technicznych związanych z logowaniem operatora.

Jednym z głównych założeń modelu jest oddzielenie danych osoby od danych biometrycznych. Dane opisowe osoby są przechowywane w tabeli `Persons`, natomiast dane wykorzystywane przez identyfikację twarzy i odcisku palca znajdują się w osobnych tabelach. Dzięki temu jedna osoba może być powiązana z wieloma rekordami biometrycznymi, a podstawowy rejestr osób nie musi przechowywać wszystkich danych technicznych związanych z biometrią.

Historia prób identyfikacji została wydzielona do tabeli `IdentificationAttempts`. Dodatkowe wyniki dopasowań kandydatów są przechowywane w tabeli `MatchResults`. Takie rozdzielenie pozwala zapisać zarówno ogólny wynik próby, jak i szczegółowe informacje o kandydatach uzyskanych podczas porównania.

Ustawienia systemowe są przechowywane w tabeli `SystemSettings`. Są one niezależne od konkretnych osób, ponieważ dotyczą konfiguracji działania aplikacji. Dane operatora aplikacji znajdują się w tabeli `Users`, która służy do obsługi technicznego logowania do systemu.

Przyjęty model danych ułatwia utrzymanie informacji, testowanie aplikacji oraz dalszą rozbudowę systemu. Oddzielne tabele dla osób, danych biometrycznych, prób identyfikacji i ustawień pozwalają rozwijać poszczególne części aplikacji bez mieszania różnych typów danych w jednej strukturze.

8.2. Opis głównych encji

Główne encje modelu danych odpowiadają najważniejszym obszarom działania aplikacji: rejestrowi osób, danym biometrycznym, historii prób, ustawieniom oraz logowaniu operatora. Zestawienie tabel przedstawiono w tabeli 8.1.

Tabela	Rola w systemie
<code>Users</code>	Przechowuje dane techniczne operatora aplikacji, takie jak nazwa użytkownika, hasło, imię i nazwisko, rola, aktywność konta oraz data utworzenia. Tabela służy do obsługi logowania.
<code>SystemSettings</code>	Przechowuje ustawienia systemowe w postaci klucza, wartości, opisu i daty aktualizacji. Tabela służy do przechowywania parametrów konfiguracyjnych.
<code>Persons</code>	Przechowuje podstawowe dane osób zarejestrowanych w systemie, w tym kod osoby, imię, nazwisko, dane opisowe, status oraz ścieżki do obrazów profilowych i biometrycznych.

Tabela	Rola w systemie
FaceTemplates	Przechowuje szablony twarzy powiązane z osobami. Zawiera m.in. identyfikator osoby, ścieżkę do obrazu źródłowego, zapis embeddingu, ocenę jakości, nazwę modelu.
FingerprintTemplates	Przechowuje szablony odcisków palców powiązane z osobami. Zawiera m.in. identyfikator osoby, pozycję palca, ścieżkę do obrazu źródłowego, dane szablonu, ocenę jakości, nazwę algorytmu, rozdzielczość DPI.
BiometricSamples	Przechowuje informacje o próbkach biometrycznych przypisanych do osób. Tabela zawiera typ biometrii, ścieżkę do pliku, ocenę jakości, notatki oraz datę utworzenia.
IdentificationAttempts	Przechowuje historię prób identyfikacji. Zawiera datę próby, typ biometrii, ścieżkę do pliku wejściowego, najlepiej dopasowaną osobę, wynik podobieństwa, decyzję, operatora oraz status próby.
MatchResults	Przechowuje wyniki kandydatów powiązane z konkretną próbą identyfikacji. Tabela zapisuje kilku wyników dopasowania, ich kolejności, decyzji oraz powiązania z osobą.

Tabela 8.1. Główne tabele bazy danych systemu IdentityCore

Model danych obejmuje zarówno tabele bezpośrednio związane z identyfikacją biometryczną, jak i tabele pomocnicze. Dzięki temu baza danych przechowuje dane osób, szablony, ustawienia, historię prób oraz wyniki kandydatów.

8.3. Relacje między tabelami

Centralną rolę w modelu danych pełni tabela `Persons`. To z nią powiązane są dane biometryczne twarzy, dane biometryczne odcisków palców oraz próbki biometryczne. Relacje te pozwalają przypisać wiele rekordów biometrycznych do jednej osoby.

Tabela `FaceTemplates` posiada klucz obcy `PersonId` wskazujący na tabelę `Persons`. Relacja ta wykorzystuje regułę `ON DELETE CASCADE`, co oznacza, że usunięcie osoby powoduje usunięcie powiązanych szablonów twarzy. Analogicznie tabela `FingerprintTemplates` posiada klucz obcy `PersonId` wskazujący na tabelę `Persons` również z regułą `ON DELETE CASCADE`.

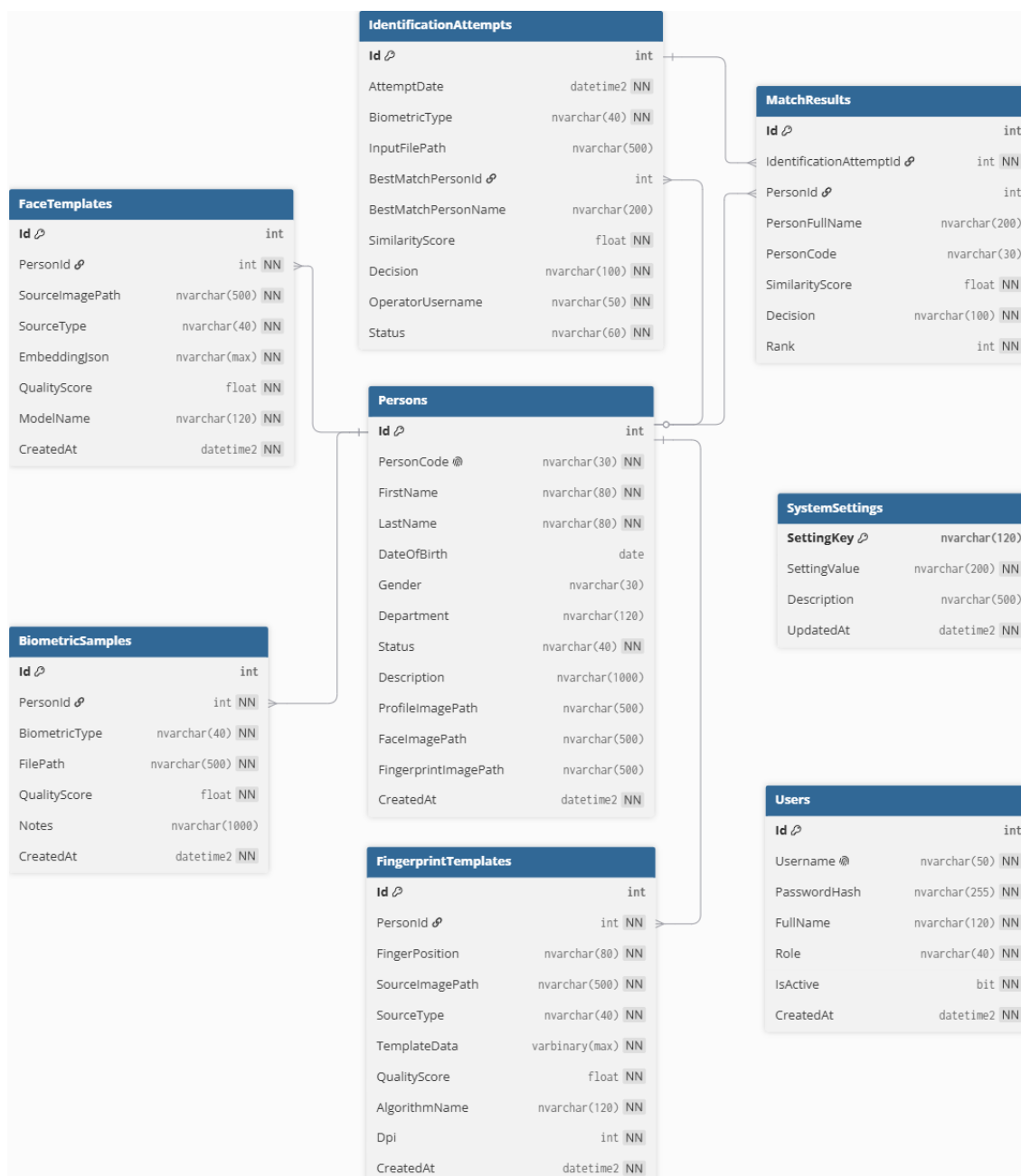
Tabela `BiometricSamples` także jest powiązana z tabelą `Persons` przez pole `PersonId`. Dzięki temu próbki biometryczne nie funkcjonują jako niezależne rekordy, lecz są przypisane do konkretnej osoby.

Tabela `IdentificationAttempts` przechowuje informacje o wykonanych próbach identyfikacji. Pole `BestMatchPersonId` wskazuje na osobę najlepiej dopasowaną podczas próby. Relacja z tabelą `Persons` wykorzystuje regułę `ON DELETE SET NULL`, dlatego usunięcie osoby nie usuwa historii próby, lecz zeruje powiązanie z osobą. Pozwala to zachować zapis historii nawet wtedy, gdy rekord osoby zostanie usunięty.

Tabela `MatchResults` jest powiązana z tabelą `IdentificationAttempts` przez pole `IdentificationAttemptId`. Zastosowano tutaj regułę `ON DELETE CASCADE`, więc usunięcie próby identyfikacji usuwa również powiązane wyniki kandydatów. Tabela `MatchResults` może być też powiązana z tabelą `Persons` przez pole `PersonId`. W tym przypadku zastosowano regułę `ON DELETE NO ACTION`.

8.4. Diagram ERD

Diagram ERD przedstawia graficzne ujęcie tabel oraz relacji występujących w modelu danych. Obejmuje on przede wszystkim powiązania tabel `Persons`, `FaceTemplates`, `FingerprintTemplates`, `BiometricSamples`, `IdentificationAttempts` oraz `MatchResults`. Tabele `SystemSettings` i `Users` pełnią funkcje pomocnicze i również zostały uwzględnione w modelu.



Rysunek 8.1. Diagram ERD bazy danych systemu IdentityCore

Diagram ułatwia zrozumienie, które tabele stanowią centrum modelu danych oraz w jaki sposób rekordy biometryczne i historia prób są powiązane z osobami zarejestrowanymi w systemie.

8.5. Integralność danych

Integralność danych w bazie wynika przede wszystkim z zastosowania kluczy głównych, kluczy obcych, ograniczeń unikalności oraz wartości domyślnych. Większość głównych tabel posiada klucz główny `Id` typu całkowitego z automatycznym nadawaniem wartości. Wyjątkiem jest tabela `SystemSettings`, w której kluczem głównym jest `SettingKey`.

W tabeli `Persons` zastosowano unikalny kod osoby zapisany w kolumnie `PersonCode`. Tabela `Users` posiada unikalną nazwę użytkownika w kolumnie `Username`. Ograniczenia te zmniejszają ryzyko duplikowania rekordów o tym samym identyfikatorze logicznym.

Klucze obce wiążą dane biometryczne z osobami oraz wyniki dopasowań z próbami identyfikacji. Tabele `FaceTemplates`, `FingerprintTemplates` i `BiometricSamples` odnoszą się do tabeli `Persons`, natomiast tabela `MatchResults` odnosi się do tabeli `IdentificationAttempts`. Dzięki temu szablony i wyniki nie są przechowywane jako całkowicie oderwane dane.

Oddzielenie danych osób od danych biometrycznych ogranicza duplikację informacji. Dane opisowe osoby nie muszą być powtarzane przy każdym szablonie twarzy lub odcisku palca. W przypadku dodania kolejnych próbek lub szablonów wystarczy utworzyć nowe rekordy powiązane z istniejącą osobą.

8.6. Skrypty bazy danych i migracje

W projekcie zastosowano trzy główne skrypty SQL przechowywane w katalogu `Database`. Ich zadaniem jest przygotowanie bazy danych, aktualizacja starszej struktury oraz czyszczenie danych biometrycznych podczas testów.

Plik `create_database.sql` służy do utworzenia bazy `IdentityCoreDb` od podstaw. Skrypt tworzy pełną aktualną strukturę tabel, usuwa wcześniej istniejące tabele aplikacji, zakłada klucze, relacje oraz indeksy, a także dodaje podstawowe dane startowe i domyślne ustawienia decyzyjne. Jest to skrypt przeznaczony do świeżej instalacji albo świadomego odtworzenia bazy od zera.

Plik `update_existing_database.sql` służy do aktualizacji starszej istniejącej bazy bez usuwania danych. Skrypt dodaje brakującą kolumnę `ProfileImagePath`, tworzy tabele `FaceTemplates`, `FingerprintTemplates` i `SystemSettings`, jeżeli ich nie ma, dodaje brakujące indeksy oraz uzupełnia ustawienia systemowe. Zastosowanie instrukcji sprawdzających istnienie tabel, kolumn i indeksów sprawia, że skrypt można traktować jako pomocniczą migrację aktualizującą starszą strukturę.

Plik `cleanup_biometric_data.sql` pełni funkcję testową i porządkową. Umożliwia usunięcie danych dotyczących twarzy i odcisków palców bez usuwania osób z rejestru. Skrypt usuwa powiązane wyniki dopasowań, próby identyfikacji, szablony biometryczne oraz próbki, a także czyści ścieżki do obrazów biometrycznych w tabeli `Persons`. Na końcu zwraca podstawowe zestawienia kontrolne, które pozwalają sprawdzić stan danych po czyszczeniu.

8.7. Uzasadnienie przyjętego modelu danych

Przyjęty model danych odpowiada potrzebom lokalnej aplikacji desktopowej służącej do identyfikacji osób. Oddzielenie danych opisowych osób od danych biometrycznych ułatwia utrzymanie struktury bazy i pozwala przechowywać wiele szablonów lub próbek dla jednej osoby.

Osobne tabele dla szablonów twarzy, szablonów odcisków palców i próbek biometrycznych pozwalają rozwijać część biometryczną bez zmiany podstawowego rejestru osób. Tabela historii prób oraz tabela wyników kandydatów umożliwiają analizę wykonanych identyfikacji, a tabela ustawień systemowych pozwala przechowywać konfigurację działania aplikacji niezależnie od kodu.

9. Implementacja aplikacji

9.1. Najważniejsze moduły systemu

Aplikacja *IdentityCore* została zaimplementowana jako lokalna aplikacja desktopowa WPF/.NET. Jej kod podzielono na części odpowiadające głównym funkcjom systemu: logowaniu operatora, rejestrowi osób, identyfikacji twarzy, identyfikacji odcisku palca, historii prób, ustawieniom systemowym oraz komunikacji z bazą danych SQL Server.

Najważniejsze moduły implementacyjne aplikacji przedstawiono w tabeli 9.1. Tabela nie opisuje po-
nownie architektury całego projektu, lecz pokazuje, które elementy kodu odpowiadają za konkretne funkcje systemu.

Moduł	Przykładowe elementy projektu	Rola w aplikacji
Logowanie operatora	LoginWindow, AuthenticationService, UserRepository	Sprawdzenie danych operatora i dopuszczenie go do głównej części aplikacji.
Rejestr osób	PersonRegistryView, PersonRegistryViewModel, PersonRepository	Dodawanie, odczyt, edycja i usuwanie profili osób zapisanych w systemie.
Identyfikacja twarzy	FaceIdentificationView, FaceIdentificationViewModel, FaceRecognitionService, FaceEmbeddingService	Obsługa próby identyfikacji osoby na podstawie obrazu twarzy i reprezentacji uzyskanej z modelu <code>arcface.onnx</code> .
Identyfikacja odcisku palca	FingerprintIdentificationView, FingerprintIdentificationViewModel, FingerprintRecognitionService	Obsługa próby identyfikacji osoby na podstawie obrazu odcisku palca i zapisanego szablonu biometrycznego.
Obsługa obrazów	ImageProcessingService, CameraService	Przygotowanie danych obrazowych oraz obsługa obrazu pochodzącego z pliku lub kamery.
Historia prób	HistoryView, HistoryViewModel, IdentificationRepository	Zapis i odczyt informacji o wykonanych próbach identyfikacji.
Ustawienia systemowe	SettingsView, SettingsViewModel, SystemSettingsService	Obsługa parametrów decyzyjnych wykorzystywanych przez moduły biometryczne.

Moduł	Przykładowe elementy projektu	Rola w aplikacji
Dostęp do danych	DatabaseConnection, PersonRepository, FaceTemplateRepository, FingerprintTemplateRepository, BiometricSampleRepository	Komunikacja z bazą danych SQL Server oraz wykonywanie operacji zapisu, odczytu, aktualizacji i usuwania danych.
Modele danych	Person, FaceIdentificationResult, FingerprintIdentificationResult, BiometricDecisionSettings, IdentificationAttempt	Reprezentacja danych przekazywanych między widokami, serwisami i repozytoriami.
Nawigacja i komendy	MainViewModel, RelayCommand	Przełączanie głównych widoków aplikacji oraz obsługa akcji wykonywanych przez operatora.

Tabela 9.1. Najważniejsze moduły implementacyjne aplikacji IdentityCore

W przedstawionym podziale widoki odpowiadają za prezentację danych, modele widoków za obsługę stanu ekranu i reakcji na działania operatora, serwisy za logikę operacji, a repozytoria za kontakt z bazą danych. Dzięki temu poszczególne funkcje aplikacji są rozdzielone i łatwiejsze do utrzymania.

9.2. Implementacja rejestracji osoby

Rejestracja osoby została zrealizowana w module rejestru osób. Za obsługę tego obszaru odpowiadają przede wszystkim `PersonRegistryView`, `PersonRegistryViewModel` oraz `PersonRepository`. Widok udostępnia formularz danych osoby, model widoku przechowuje aktualnie wybrany profil i obsługuje akcje operatora, a repozytorium wykonuje zapis i odczyt danych z bazy.

Dodanie nowej osoby odbywa się w metodzie `AddNewPerson`. Tworzony jest nowy obiekt `Person`, któremu nadawany jest kod osoby, podstawowe dane domyślne, status oraz puste ścieżki do obrazów. Następnie obiekt jest przekazywany do metody `Add` w klasie `PersonRepository`. Po zapisie do bazy metoda zwraca identyfikator nowo utworzonego rekordu, który jest przypisywany do obiektu w aplikacji.

Edycja osoby jest realizowana przez metodę `SaveChanges`. Jeżeli istnieje aktualnie wybrany profil, model widoku przekazuje go do metody `Update` w `PersonRepository`. Po aktualizacji dane są ponownie wczytywane, aby lista osób i szczegóły wybranego profilu odpowiadały stanowi zapisanemu w bazie danych.

Podstawowa kontrola poprawności operacji polega między innymi na sprawdzaniu, czy wybrano osobę, na której można wykonać daną akcję. Dotyczy to zapisu zmian, usuwania profilu, ustawiania zdjęcia profilowego oraz rejestrowania danych biometrycznych. Kompletność pól wymaganych jest dodatkowo wymuszana przez model danych i strukturę bazy, której szczegóły opisano w rozdziale 8.

W aplikacji zaimplementowano również obsługę zdjęcia profilowego. Metoda `SetProfileImage` umożliwia wybór pliku graficznego, skopiowanie go do lokalnego katalogu roboczego aplikacji tworzonego w katalogu uruchomieniowym programu, zapisanie ścieżki w polu `ProfileImagePath` oraz aktualizację osoby w bazie danych. Usunięcie zdjęcia profilowego realizuje metoda `RemoveProfileImage`, która czyści ścieżkę i zapisuje zmieniony profil.

Dane biometryczne osoby są powiązane z profilem osoby, ale nie są zapisywane jako zwykłe pola formularza opisowego. Rejestracja próbek twarzy i odcisków palców jest obsługiwana przez osobne metody oraz repozytoria biometryczne, między innymi `FaceTemplateRepository`, `FingerprintTemplateRepository` i `BiometricSampleRepository`. Dzięki temu profil osoby pozostaje oddzielony od danych technicznych używanych w procesie identyfikacji.

9.3. Implementacja procesu biometrycznego

Proces biometryczny w aplikacji *IdentityCore* został zaimplementowany jako przepływ danych pomiędzy modelem widoku, usługami biometrycznymi, repozytoriami oraz obiektami wynikowymi. Widok i model widoku odpowiadają za przyjęcie danych od operatora i pokazanie rezultatu, natomiast właściwa analiza biometryczna odbywa się w serwisach. Szczegóły metod identyfikacji zostały opisane w rozdziale 5; w tym miejscu przedstawiono ich praktyczne użycie w kodzie aplikacji.

9.3.1. Implementacja identyfikacji twarzy

Identyfikacja twarzy rozpoczyna się w `FaceIdentificationViewModel`. Model widoku obsługuje komendy związane z kamerą, wczytaniem obrazu, rozpoczęciem identyfikacji, zapisem próby oraz otwarciem profilu osoby. Obraz wejściowy może pochodzić z pliku wskazanego przez operatora albo z kamery obsługiwanej przez `CameraService`. W przypadku kamery aplikacja pobiera klatkę, zapisuje ją jako plik wejściowy i dopiero taki obraz przekazuje do dalszej analizy.

Po uruchomieniu identyfikacji `FaceIdentificationViewModel` wywołuje usługę `FaceRecognitionService`. Model widoku nie wykonuje samodzielnie porównywania twarzy; jego zadaniem jest przekazanie ścieżki obrazu, obsługa stanu analizy, odebranie wyniku i przepisanie go do właściwości prezentowanych w interfejsie.

Pierwszym ważnym etapem po stronie kodu jest przygotowanie obrazu twarzy. Odpowiada za to `ImageProcessingService`. Fragment pokazujący przygotowanie próbki twarzy przedstawiono na listingu 9.1.

Listing 9.1. Przygotowanie obrazu twarzy przed analizą

```
1 public Mat? ExtractAndPrepareFace(Mat originalFrame, out double qualityScore)
2 {
3     qualityScore = 0;
4
5     if (originalFrame.Empty())
6     {
7         return null;
8     }
9
10    var faceRect = DetectFrontalFace(originalFrame);
11
12    if (faceRect.Width == 0 || faceRect.Height == 0)
13    {
14        return null;
15    }
16
17    var expandedFaceRect = ExpandToSquareRect(faceRect, originalFrame.Width,
18        originalFrame.Height, 0.28);
19
20    using var faceCrop = new Mat(originalFrame, expandedFaceRect).Clone();
```

```

21     if (!IsCropGeometryUsable(faceCrop))
22     {
23         return null;
24     }
25
26     qualityScore = EstimateImageQuality(faceCrop);
27
28     var resized = new Mat();
29     Cv2.Resize(faceCrop, resized, new Size(TargetSize, TargetSize), 0, 0,
        InterpolationFlags.Area);
30
31     if (!ContainsFrontalFace(resized))
32     {
33         resized.Dispose();
34         return null;
35     }
36
37     return resized;
38 }

```

W pokazanym fragmencie aplikacja nie przekazuje całego kadru bezpośrednio do modelu. Najpierw wykrywa twarz, przygotowuje wycinek obrazu, sprawdza jego użyteczność, ocenia jakość i skaluje próbkę do wymaganego rozmiaru. Jeżeli nie uda się uzyskać poprawnego obrazu twarzy, dalsza identyfikacja nie jest wykonywana.

Po przygotowaniu próbki wykorzystywany jest `FaceEmbeddingService`. To w tej usłudze aplikacja używa pliku `arcface.onnx` i zamienia przygotowany obraz na reprezentację używaną później przy porównywaniu. Fragment tej części implementacji przedstawiono na listingu 9.2.

Listing 9.2. Użycie modelu `arcface.onnx` do wygenerowania embeddingu twarzy

```

1 private const int ImageSize = 112;
2 private const string ModelFileName = "arcface.onnx";
3
4 private readonly MLContext _mlContext = new(seed: 1);
5 private readonly PredictionEngine<FaceOnnxInput, FaceOnnxOutput> _predictionEngine;
6 private readonly object _predictionLock = new();
7
8 public FaceEmbeddingService()
9 {
10     var modelPath = ResolveModelPath();
11
12     if (!File.Exists(modelPath))
13     {
14         throw new FileNotFoundException(
15             "Nie znaleziono modelu arcface.onnx. Sprawdź folder MLModels i ustawienie
                Copy to Output Directory.",
16             modelPath);
17     }
18
19     var emptyData = _mlContext.Data.LoadFromEnumerable(new List<FaceOnnxInput>());
20
21     var pipeline = _mlContext.Transforms.ApplyOnnxModel(
22         outputColumnNames: new[] { "embedding" },
23         inputColumnNames: new[] { "input_1" },
24         modelFile: modelPath);
25

```

```

26     var model = pipeline.Fit(emptyData);
27
28     _predictionEngine = _mlContext.Model.CreatePredictionEngine<FaceOnnxInput,
        FaceOnnxOutput>(model);
29 }
30
31 public float[] GenerateEmbeddingFromMat(Mat preparedFaceBgr)
32 {
33     var input = CreateOnnxInput(preparedFaceBgr);
34
35     FaceOnnxOutput output;
36     lock (_predictionLock)
37     {
38         output = _predictionEngine.Predict(input);
39     }
40
41     return NormalizeEmbedding(output.Embedding);
42 }

```

Listing pokazuje miejsce, w którym aplikacja odnajduje model `arcface.onnx`, tworzy potok ONNX i uruchamia predykcję dla przygotowanego obrazu twarzy. Wynikiem działania tej usługi jest reprezentacja przekazywana dalej do `FaceRecognitionService`.

Kolejnym etapem jest pobranie zapisanych wzorców twarzy i utworzenie rankingu kandydatów. Wykorzystywane są tutaj `FaceTemplateRepository` oraz `PersonRepository`. Fragment odpowiedzialny za ten etap przedstawiono na listingu 9.3.

Listing 9.3. Wybór kandydatów podczas identyfikacji twarzy

```

1  var templates = _faceTemplateRepository.GetAll();
2
3  if (templates.Count == 0)
4  {
5      return CreateFailedResult(
6          "W bazie nie ma zarejestrowanych szablonów twarzy. Najpierw zarejestruj próbki
            twarzy w profilu osoby.",
7          "Nie znaleziono dopasowania",
8          matchThreshold,
9          minimumMargin);
10 }
11
12 var persons = _personRepository
13     .GetAll()
14     .ToDictionary(person => person.Id);
15
16 var rawScores = new List<RawFaceCandidateScore>();
17
18 foreach (var template in templates)
19 {
20     if (!persons.TryGetValue(template.PersonId, out var person))
21     {
22         continue;
23     }
24
25     var templateEmbedding = DeserializeEmbedding(template.EmbeddingJson);
26
27
28

```

```
29     if (templateEmbedding.Length == 0)
30     {
31         continue;
32     }
33
34     var similarity = CalculateSimilarityPercentage(inputEmbedding, templateEmbedding);
35
36     rawScores.Add(new RawFaceCandidateScore
37     {
38         Person = person,
39         Template = template,
40         SimilarityScore = similarity
41     });
42 }
43
44 if (rawScores.Count == 0)
45 {
46     return CreateFailedResult(
47         "Nie udało się odczytać żadnego poprawnego szablonu twarzy z bazy.",
48         "Nie znaleziono dopasowania",
49         matchThreshold,
50         minimumMargin);
51 }
52
53 var candidates = rawScores
54     .GroupBy(score => score.Person.Id)
55     .Select(group => BuildCandidateForPerson(group.ToList()))
56     .OrderByDescending(candidate => candidate.SimilarityScore)
57     .Take(3)
58     .Select((candidate, index) =>
59     {
60         candidate.Rank = index + 1;
61         return candidate;
62     })
63     .ToList();
```

W tym fragmencie aplikacja pobiera zapisane szablony twarzy, łączy je z osobami z rejestru, odczytuje zapisane reprezentacje i porównuje je z próbką wejściową. Wyniki są grupowane według osoby, ponieważ jedna osoba może posiadać więcej niż jeden zapisany szablon. Następnie tworzony jest ranking najlepszych kandydatów.

Samo porównanie dwóch reprezentacji twarzy zostało wydzielone do osobnego fragmentu. Listing 9.4 pokazuje implementację obliczenia wyniku podobieństwa wykorzystywanego podczas budowania rankingu kandydatów.

Listing 9.4. Obliczanie podobieństwa cosinusowego dla embeddingów twarzy

```
1 private static double CalculateSimilarityPercentage(float[] first, float[] second)
2 {
3     if (first.Length == 0 || second.Length == 0 || first.Length != second.Length)
4     {
5         return 0;
6     }
7
8     double dot = 0;
9     double normFirst = 0;
10    double normSecond = 0;
```

```

11
12     for (var i = 0; i < first.Length; i++)
13     {
14         dot += first[i] * second[i];
15         normFirst += first[i] * first[i];
16         normSecond += second[i] * second[i];
17     }
18
19     if (normFirst == 0 || normSecond == 0)
20     {
21         return 0;
22     }
23
24     var cosine = dot / (Math.Sqrt(normFirst) * Math.Sqrt(normSecond));
25     var percentage = cosine * 100.0;
26
27     return Math.Clamp(percentage, 0, 100);
28 }

```

W pokazanym fragmencie kod sprawdza poprawność porównywanych wektorów, oblicza wynik podobieństwa i ogranicza go do zakresu procentowego. Wartość zwrócona przez tę metodę jest później używana przy wyborze najlepszego kandydata i ocenie, czy wynik spełnia wymagania decyzyjne.

Po utworzeniu listy kandydatów `FaceRecognitionService` wybiera najlepszy wynik i porównuje go z drugim kandydatem. W tym miejscu stosowane są ustawienia decyzyjne pobrane z `SystemSettingsService`, czyli próg dopasowania oraz minimalny margines. Jeżeli w bazie nie ma wzorców twarzy, nie uda się odczytać poprawnych danych, na obrazie nie wykryto twarzy albo wynik jest zbyt słaby, aplikacja zwraca decyzję o braku dopasowania wraz z komunikatem opisującym problem.

9.3.2. Implementacja identyfikacji odcisku palca

Identyfikacja odcisku palca rozpoczyna się w `FingerprintIdentificationViewModel`. Model widoku obsługuje wczytanie obrazu odcisku z pliku, rozpoczęcie analizy, zapis próby oraz otwarcie profilu osoby. Po wybraniu pliku ścieżka obrazu jest zapisywana jako `InputFilePath`, a następnie przekazywana do `FingerprintRecognitionService`.

Właściwa logika biometryczna znajduje się w `FingerprintRecognitionService`. Pierwszym kluczowym etapem jest utworzenie szablonu próbki wejściowej. Fragment odpowiedzialny za zamianę obrazu odcisku na dane szablonu przedstawiono na listingu 9.5.

Listing 9.5. Utworzenie szablonu odcisku palca na podstawie obrazu wejściowego

```

1 public FingerprintTemplateExtractionResult CreateTemplate(string imagePath, int dpi =
   DefaultDpi)
2 {
3     try
4     {
5         var resolvedPath = ResolveImagePath(imagePath);
6
7         if (string.IsNullOrEmpty(resolvedPath) || !File.Exists(resolvedPath))
8         {
9             return new FingerprintTemplateExtractionResult
10             {
11                 Success = false,
12                 Message = "Nie znaleziono obrazu odcisku palca."
13             };

```

```
14     }
15
16     var imageBytes = File.ReadAllBytes(resolvedPath);
17
18     var image = new FingerprintImage(
19         imageBytes,
20         new FingerprintImageOptions
21         {
22             Dpi = dpi
23         });
24
25     var template = new SourceAFIS.FingerprintTemplate(image);
26     var templateBytes = template.ToArray();
27
28     if (templateBytes.Length == 0)
29     {
30         return new FingerprintTemplateExtractionResult
31         {
32             Success = false,
33             Message = "Nie udało się utworzyć szablonu cech odcisku palca."
34         };
35     }
36
37     return new FingerprintTemplateExtractionResult
38     {
39         Success = true,
40         Message = "Szablon cech odcisku palca został utworzony przez SourceAFIS.",
41         TemplateData = templateBytes,
42         SourceImagePath = imagePath,
43         QualityScore = EstimateTemplateQuality(templateBytes),
44         AlgorithmName = "SourceAFIS",
45         Dpi = dpi
46     };
47 }
48 catch (Exception ex)
49 {
50     return new FingerprintTemplateExtractionResult
51     {
52         Success = false,
53         Message = $"Błąd tworzenia szablonu cech odcisku: {ex.Message}"
54     };
55 }
56 }
```

W pokazanym fragmencie aplikacja sprawdza, czy plik obrazu istnieje, odczytuje jego zawartość i tworzy szablon próbki wejściowej. Jeżeli plik nie zostanie znaleziony, nie można go odczytać albo szablon jest pusty, metoda zwraca wynik niepowodzenia z komunikatem.

Po utworzeniu szablonu próbki aplikacja pobiera zapisane szablony odcisków palców przez `FingerprintTemplateRepository` oraz dane osób przez `PersonRepository`. Następnie próbka wejściowa jest porównywana z kolejnymi wzorcami zapisanymi w bazie. Fragment tej części procesu przedstawiono na listingu 9.6.

Listing 9.6. Porównywanie próbki odcisku palca z zapisanymi szablony

```
1 var probeTemplate = new SourceAFIS.FingerprintTemplate(probeExtraction.TemplateData);
2 var matcher = new FingerprintMatcher(probeTemplate);
3
4 var rawCandidates = new List<FingerprintCandidateMatch>();
5
6 foreach (var storedTemplate in storedTemplates)
7 {
8     if (!persons.TryGetValue(storedTemplate.PersonId, out var person))
9     {
10         continue;
11     }
12
13     if (storedTemplate.TemplateData.Length == 0)
14     {
15         continue;
16     }
17
18     var candidateTemplate = new SourceAFIS.FingerprintTemplate(storedTemplate.
19         TemplateData);
20     var score = matcher.Match(candidateTemplate);
21
22     rawCandidates.Add(new FingerprintCandidateMatch
23     {
24         PersonId = person.Id,
25         PersonFullName = person.FullName,
26         PersonCode = person.PersonCode,
27         Department = person.Department,
28         SourceImagePath = storedTemplate.SourceImagePath,
29         SimilarityScore = Math.Round(score, 2)
30     });
31 }
```

Listing pokazuje, jak aplikacja tworzy matcher dla próbki wejściowej, przechodzi po zapisanych szablony, pomija rekordy bez osoby albo bez danych szablonu i wylicza wynik dopasowania. Dla każdego poprawnego porównania tworzony jest obiekt kandydata zawierający dane osoby oraz wynik podobieństwa.

Po zebraniu wyników aplikacja wybiera najlepszych kandydatów i ocenia, czy wynik może zostać zaakceptowany. Fragment logiki decyzyjnej przedstawiono na listingu 9.7.

Listing 9.7. Decyzja identyfikacyjna na podstawie progu i marginesu dopasowania

```
1 var best = candidates[0];
2 var secondScore = candidates.Count > 1 ? candidates[1].SimilarityScore : 0;
3
4 var margin = candidates.Count > 1
5     ? Math.Round(best.SimilarityScore - secondScore, 2)
6     : double.PositiveInfinity;
7
8 var scoreAccepted = best.SimilarityScore >= matchThreshold;
9 var marginAccepted = candidates.Count <= 1 || margin >= minimumMatchMargin;
10 var isMatch = scoreAccepted && marginAccepted;
```

W tym fragmencie najlepszy wynik jest porównywany z wymaganym progiem dopasowania, a następnie sprawdzana jest różnica względem drugiego kandydata. Jeżeli wynik jest zbyt niski albo przewaga nad kolejnym kandydatem jest niewystarczająca, próba nie zostaje uznana za dopasowanie. Do modelu widoku trafia wtedy wynik negatywny wraz z informacją diagnostyczną.

Wynik działania `FingerprintRecognitionService` jest przekazywany z powrotem do `FingerprintIdentificationViewModel`. Model widoku aktualizuje dane najlepszego kandydata, wynik, decyzję, komunikat, próg, margines oraz listę kandydatów. Dzięki temu warstwa interfejsu otrzymuje gotowy rezultat próby bez wykonywania logiki porównywania w samym widoku.

9.3.3. Zapis wyniku identyfikacji w historii

Po zakończeniu próby identyfikacji aplikacja przygotowuje obiekt `IdentificationAttempt`. Jest on tworzony po stronie modelu widoku na podstawie wyniku zwróconego przez serwis biometryczny. Do obiektu trafiają dane opisujące próbę, między innymi typ biometrii, ścieżka pliku wejściowego, najlepiej dopasowana osoba, wynik podobieństwa, decyzja, operator, status oraz data wykonania próby.

Sam zapis do bazy danych wykonuje `IdentificationRepository`. Fragment metody zapisującej próbę identyfikacji przedstawiono na listingu 9.8.

Listing 9.8. Zapis próby identyfikacji do historii

```
1 public int AddAttempt(IdentificationAttempt attempt)
2 {
3     using var connection = DatabaseConnection.CreateConnection();
4     connection.Open();
5
6     const string query = @"
7     INSERT INTO dbo.IdentificationAttempts
8     (
9         AttemptDate,
10        BiometricType,
11        InputFilePath,
12        BestMatchPersonId,
13        BestMatchPersonName,
14        SimilarityScore,
15        Decision,
16        OperatorUsername,
17        Status
18    )
19    OUTPUT INSERTED.Id
20    VALUES
21    (
22        @AttemptDate,
23        @BiometricType,
24        @InputFilePath,
25        @BestMatchPersonId,
26        @BestMatchPersonName,
27        @SimilarityScore,
28        @Decision,
29        @OperatorUsername,
30        @Status
31    );
32 ";
33
34
```

```

35     using var command = new SqlCommand(query, connection);
36
37     command.Parameters.AddWithValue("@AttemptDate", attempt.AttemptDate);
38     command.Parameters.AddWithValue("@BiometricType", attempt.BiometricType);
39     command.Parameters.AddWithValue("@InputFilePath", string.IsNullOrEmpty(attempt.
        InputFilePath) ? DBNull.Value : attempt.InputFilePath);
40     command.Parameters.AddWithValue("@BestMatchPersonId", attempt.BestMatchPersonId.
        HasValue ? attempt.BestMatchPersonId.Value : DBNull.Value);
41     command.Parameters.AddWithValue("@BestMatchPersonName", string.IsNullOrEmpty(
        attempt.BestMatchPersonName) ? DBNull.Value : attempt.BestMatchPersonName);
42     command.Parameters.AddWithValue("@SimilarityScore", attempt.SimilarityScore);
43     command.Parameters.AddWithValue("@Decision", attempt.Decision);
44     command.Parameters.AddWithValue("@OperatorUsername", attempt.OperatorUsername);
45     command.Parameters.AddWithValue("@Status", attempt.Status);
46
47     return Convert.ToInt32(command.ExecuteScalar());
48 }

```

W pokazanym fragmencie repozytorium tworzy polecenie SQL i przekazuje do niego wartości z obiektu `IdentificationAttempt`. Po wykonaniu zapytania zwracany jest identyfikator nowo zapisanego rekordu. Mechanizm ten pozwala później odtworzyć wykonane próby i wykorzystać je podczas analizy skuteczności opisanej w rozdziale testowym.

9.4. Implementacja logowania i autoryzacji

Logowanie operatora zostało zrealizowane jako prosta część techniczna aplikacji. Użytkownik najpierw trafia do okna `LoginWindow`, gdzie podaje dane logowania. Dopiero po poprawnym uwierzytelnieniu otwierana jest główna część systemu.

Za sprawdzenie danych odpowiada `AuthenticationService`. Usługa korzysta z `UserRepository`, które wyszukuje aktywnego użytkownika w bazie danych na podstawie nazwy użytkownika. Jeżeli użytkownik nie istnieje, jest nieaktywny albo hasło nie pasuje do wartości zapisanej w bazie, logowanie kończy się niepowodzeniem.

Fragment logiki sprawdzania danych logowania przedstawiono na listingu 9.9.

Listing 9.9. Sprawdzenie danych logowania operatora

```

1 public bool Login(string username, string password)
2 {
3     if (string.IsNullOrEmpty(username) || string.IsNullOrEmpty(password)) {
4         return false;
5     }
6
7     var user = _userRepository.GetActiveUserByUsername(username.Trim());
8
9     if (user == null) {
10         return false;
11     }
12
13     if (user.PasswordHash != password) {
14         return false;
15     }
16
17     CurrentUser = user;
18     return true;
19 }

```

W pokazanym fragmencie metoda odrzuca puste dane, pobiera aktywnego użytkownika z bazy i porównuje podane hasło z wartością zapisaną w rekordzie użytkownika. Po poprawnym sprawdzeniu ustawiany jest aktualny użytkownik aplikacji.

W aktualnej postaci logowanie ma charakter demonstracyjny i techniczny. W kodzie hasło jest porównywane z wartością przechowywaną w polu `PasswordHash`, ale nie zaimplementowano rozbudowanego mechanizmu bezpieczeństwa typowego dla systemów produkcyjnych. Jest to wystarczające dla lokalnej aplikacji projektowej, której celem jest prezentacja działania systemu identyfikacji biometrycznej.

Dane konta testowego oraz instrukcja uruchomienia aplikacji nie są rozwijane w tym miejscu, ponieważ zostaną opisane w rozdziale dotyczącym uruchomienia projektu.

9.5. Operacje CRUD

Operacje CRUD są wykonywane głównie w module rejestru osób. Klasa `PersonRepository` zawiera metody `GetAll`, `Add`, `Update`, `GetById` oraz `Delete`. Dzięki temu aplikacja może odczytywać listę osób, dodawać nowe profile, aktualizować dane, pobierać pojedynczą osobę oraz usuwać rekord osoby z bazy.

Odczyt listy osób odbywa się w metodzie `LoadPersons` w `PersonRegistryViewModel`. Metoda czyści aktualną kolekcję `Persons`, pobiera dane przez `PersonRepository.GetAll`, a następnie uzupełnia kolekcję używaną przez widok. Po wczytaniu danych aktualizowane są również liczniki osób i danych biometrycznych.

Dodanie osoby jest realizowane przez `AddNewPerson`, natomiast zapis zmian przez `SaveChanges`. Usuwanie profilu obsługuje metoda `DeleteSelectedPerson`, która wywołuje `PersonRepository.Delete`, usuwa obiekt z kolekcji oraz aktualizuje wybór aktualnej osoby.

Operacje związane z ustawieniami systemowymi są realizowane przez `SettingsViewModel` oraz `SystemSettingsService`. Pozwalają one odczytywać i zapisywać wartości parametrów decyzyjnych bez modyfikowania kodu aplikacji.

Historia prób identyfikacji jest obsługiwana przez `IdentificationRepository`. Repozytorium zawiera metody służące do pobierania prób oraz zapisywania nowej próby identyfikacji. Dzięki temu wyniki pracy modułów biometrycznych nie znikają po zakończeniu operacji, lecz są utrwalane w bazie danych.

W tym rozdziale nie przedstawiono pełnych zapytań SQL dla każdej operacji, ponieważ struktura bazy i skrypty zostały omówione w rozdziale 8. W kontekście implementacji istotne jest to, że operacje na danych są wykonywane przez repozytoria, a nie bezpośrednio w widokach WPF.

9.6. Nawigacja i przepływ danych w aplikacji

Nawigacja w aplikacji jest obsługiwana przez `MainViewModel`. Klasa ta przechowuje aktualnie wybrany model widoku w polu `CurrentViewModel`. Na tej podstawie główne okno aplikacji wyświetla odpowiedni obszar systemu.

Przełączanie widoków odbywa się przez komendy, między innymi `ShowDashboardCommand`, `ShowPersonRegistryCommand`, `ShowFaceIdentificationCommand`, `ShowFingerprintIdentificationCommand`, `ShowHistoryCommand` oraz `ShowSettingsCommand`. Każda z nich ustawia właściwy model widoku, aktywny klucz strony, tytuł oraz podtytuł aktualnego ekranu.

W aplikacji wykorzystywana jest klasa `RelayCommand`, która pozwala powiązać akcje interfejsu z metodami modeli widoków. Dzięki temu logika działania nie musi być umieszczana bezpośrednio w plikach XAML. Widok wywołuje komendę, a model widoku wykonuje odpowiednią operację.

Przepływ danych polega na tym, że widok prezentuje dane z modelu widoku, a model widoku korzysta z serwisów i repozytoriów. Po wykonaniu operacji dane są odświeżane, na przykład przez ponowne wczytanie listy osób albo aktualizację wyników identyfikacji. Komunikaty zwrotne są przekazywane przez właściwości takie jak `ErrorMessage` i `StatusMessage`.

Z poziomu `MainViewModel` możliwe jest również otwarcie profilu konkretnej osoby w rejestrze. Metoda `OpenPersonProfile` przełącza aplikację do rejestru osób i wskazuje wybrany profil na podstawie identyfikatora.

10. Prezentacja warstwy użytkowej

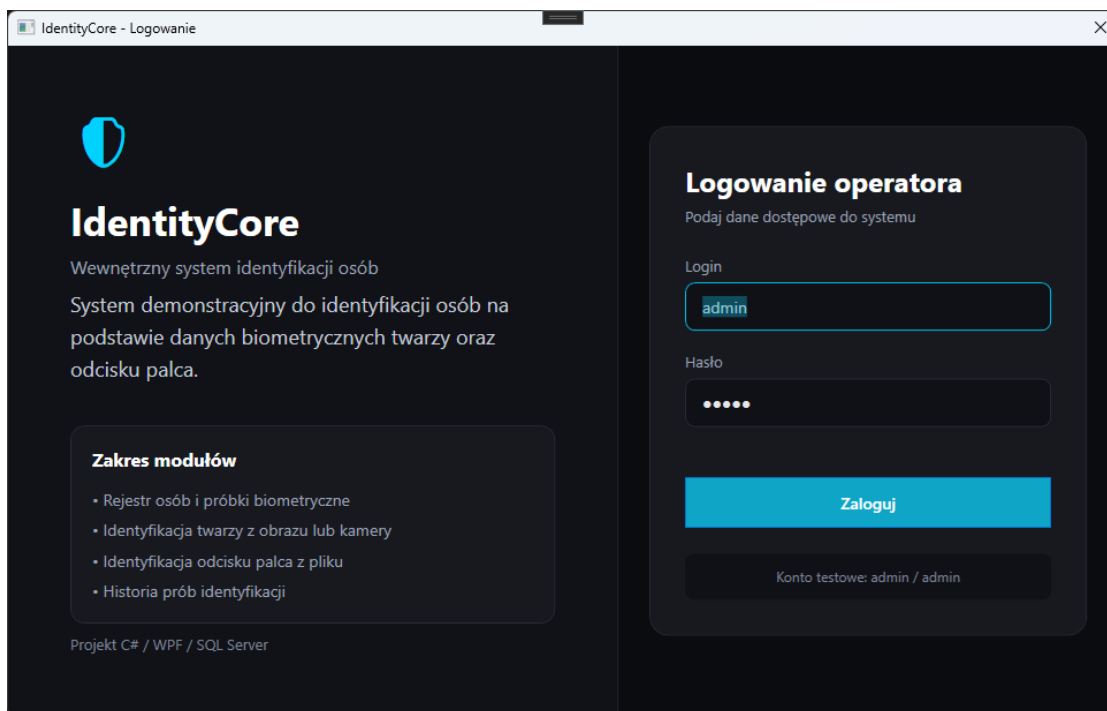
10.1. Ogólny układ interfejsu

Warstwa użytkowa aplikacji *IdentityCore* została przygotowana jako lokalny interfejs desktopowy utrzymany w ciemnym motywie. Po zalogowaniu większość widoków korzysta z tego samego układu: po lewej stronie znajduje się pionowe menu nawigacyjne z nazwą aplikacji, a po prawej główny obszar roboczy danego modułu. Aktualnie wybrany moduł jest wyróżniony innym tłem oraz akcentem kolorystycznym, dzięki czemu operator od razu widzi, w której części systemu aktualnie pracuje.

Wspólne menu boczne umożliwia przejście do panelu głównego, rejestru osób, identyfikacji twarzy, identyfikacji odcisku palca, historii prób oraz ustawień. W górnej części obszaru roboczego znajduje się tytuł widoku, krótki opis funkcji modułu oraz przycisk Wyloguj. Taki układ porządkuje sposób korzystania z aplikacji: operator najpierw wybiera moduł z menu, następnie pracuje w jego głównym panelu, a wyniki lub komunikaty są prezentowane w wydzielonych sekcjach widoku.

10.2. Ekran logowania

Pierwszym ekranem aplikacji jest ekran logowania. Jego zadaniem jest ograniczenie dostępu do głównej części systemu tylko do operatora posiadającego dane dostępowe. Po lewej stronie ekranu pokazano nazwę aplikacji, jej ogólne przeznaczenie oraz zakres dostępnych modułów. Po prawej stronie znajduje się formularz logowania z polami Login i Hasło oraz przyciskiem Zaloguj. Widoczna jest również informacja o koncie testowym, która ułatwia uruchomienie aplikacji w środowisku demonstracyjnym.

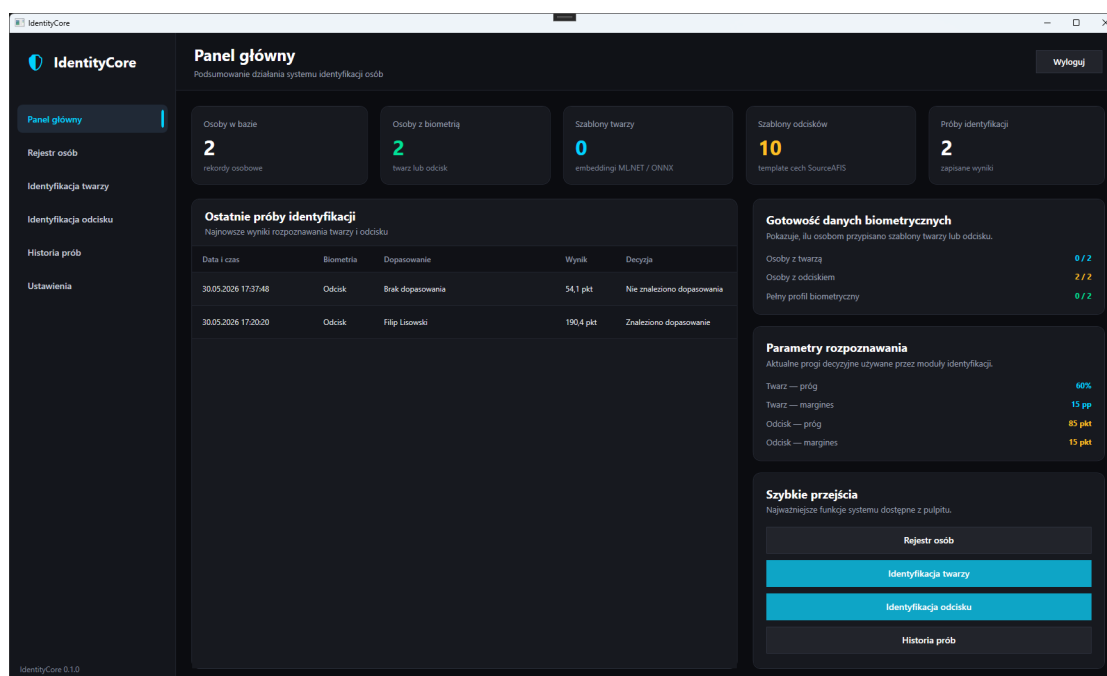


Rysunek 10.1. Ekran logowania do aplikacji IdentityCore

10.3. Panel główny aplikacji

Po poprawnym zalogowaniu operator trafia do panelu głównego. Jest to ekran podsumowujący aktualny stan systemu oraz najważniejsze dane zapisane w aplikacji. W górnej części panelu znajdują się kafle informujące o liczbie osób w bazie, liczbie osób z danymi biometrycznymi, liczbie szablonów twarzy, liczbie szablonów odcisków oraz liczbie zapisanych prób identyfikacji. Dzięki temu operator od razu widzi, czy system posiada dane potrzebne do wykonywania identyfikacji.

Centralna część panelu głównego prezentuje ostatnie próby identyfikacji. W tabeli widoczna jest data i czas próby, typ biometrii, dopasowanie, wynik oraz decyzja systemu. Prawa część widoku pokazuje gotowość danych biometrycznych, aktualne parametry rozpoznawania oraz szybkie przejścia do najważniejszych funkcji. Z tego miejsca operator może szybko przejść do rejestru osób, identyfikacji twarzy, identyfikacji odcisku lub historii prób, bez konieczności ręcznego wyszukiwania odpowiedniej funkcji w systemie.

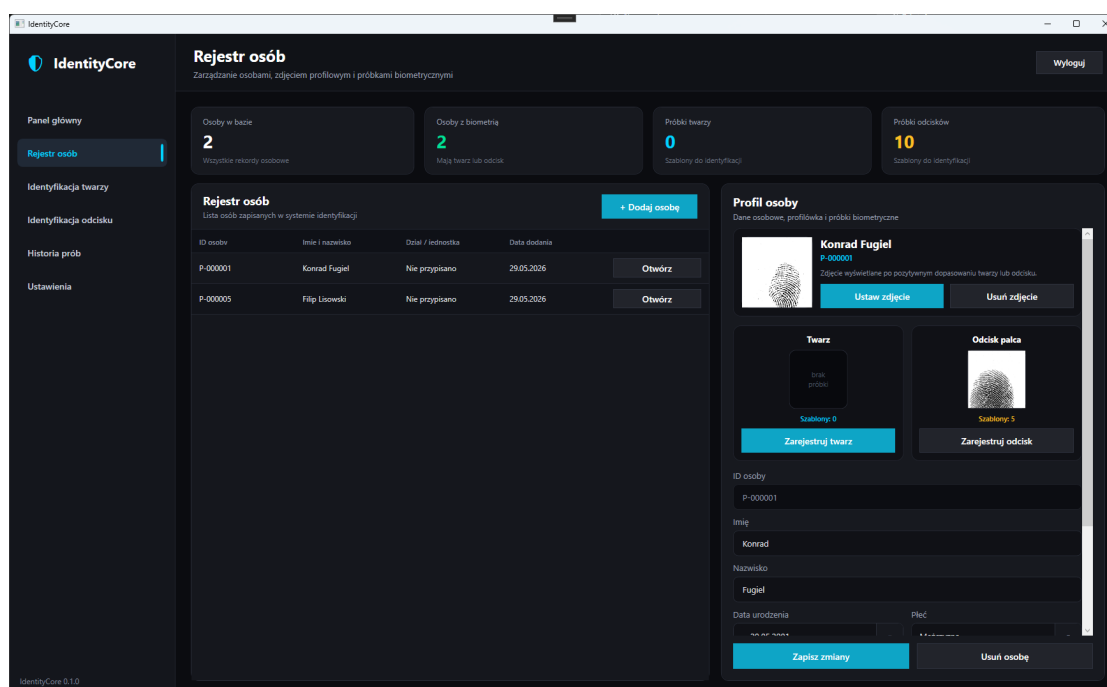


Rysunek 10.2. Panel główny aplikacji po zalogowaniu

10.4. Rejestr osób i profil osoby

Widok rejestru osób łączy listę osób zapisanych w systemie z panelem szczegółów aktualnie wybranego profilu. W górnej części widoku znajdują się kafle podsumowujące liczbę osób w bazie, liczbę osób z przypisanymi danymi biometrycznymi oraz liczbę próbek twarzy i odcisków. Poniżej, po lewej stronie, znajduje się tabela osób zawierająca identyfikator osoby, imię i nazwisko, dział lub jednostkę oraz datę dodania. Z poziomu tabeli operator może otworzyć wybrany profil.

Prawa część widoku przedstawia panel profilu osoby. Zawiera zdjęcie profilowe, podstawowe dane osoby, informacje o próbkach twarzy i odcisku palca oraz przyciski umożliwiające rejestrację danych biometrycznych. W tym samym obszarze znajdują się pola edycji danych osobowych oraz przyciski **Zapisz zmiany** i **Usuń osobę**. Dzięki temu widok rejestru pełni jednocześnie funkcję listy osób, panelu szczegółów oraz miejsca zarządzania podstawowymi danymi osoby i jej materiałem biometrycznym.

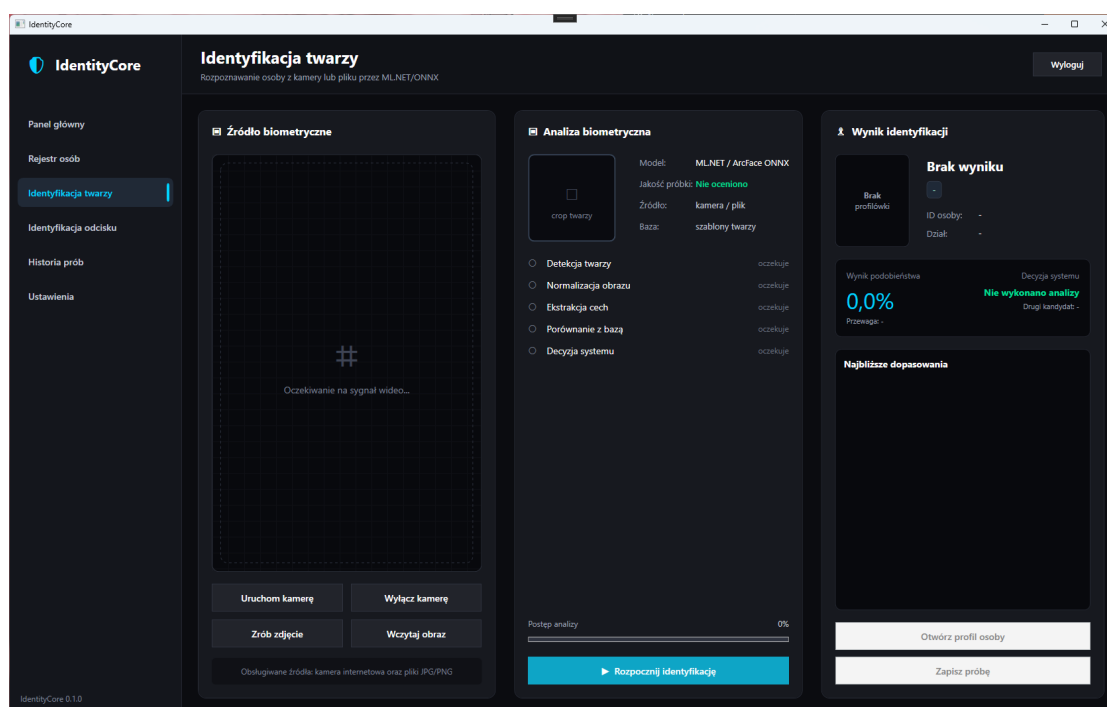


Rysunek 10.3. Rejestr osób wraz z panelem profilu wybranej osoby

10.5. Widok identyfikacji twarzy

Widok identyfikacji twarzy został podzielony na trzy główne obszary: źródło biometryczne, analizę biometryczną oraz wynik identyfikacji. Po lewej stronie znajduje się panel z podglądem obrazu oraz przyciskami **Uruchom kamerę**, **Wyłącz kamerę**, **Zrób zdjęcie** i **Wczytaj obraz**. Oznacza to, że operator może skorzystać zarówno z kamery, jak i z wcześniej przygotowanego pliku graficznego. Widok jasno informuje, że obsługiwane źródła to kamera internetowa oraz pliki obrazów.

Środkowa część widoku prezentuje przebieg analizy. Pokazane są informacje o modelu, jakości próbki, źródle danych oraz bazie wzorców. Poniżej znajduje się lista etapów, takich jak detekcja twarzy, normalizacja obrazu, ekstrakcja cech, porównanie z bazą oraz decyzja systemu. Na dole widoczny jest pasek postępu i przycisk uruchamiający identyfikację. Prawy panel służy do prezentacji wyniku: pokazuje najlepsze dopasowanie, dane osoby, wynik podobieństwa, decyzję systemu oraz listę najbliższych dopasowań. W stanie początkowym aplikacja informuje, że analiza nie została jeszcze wykonana.

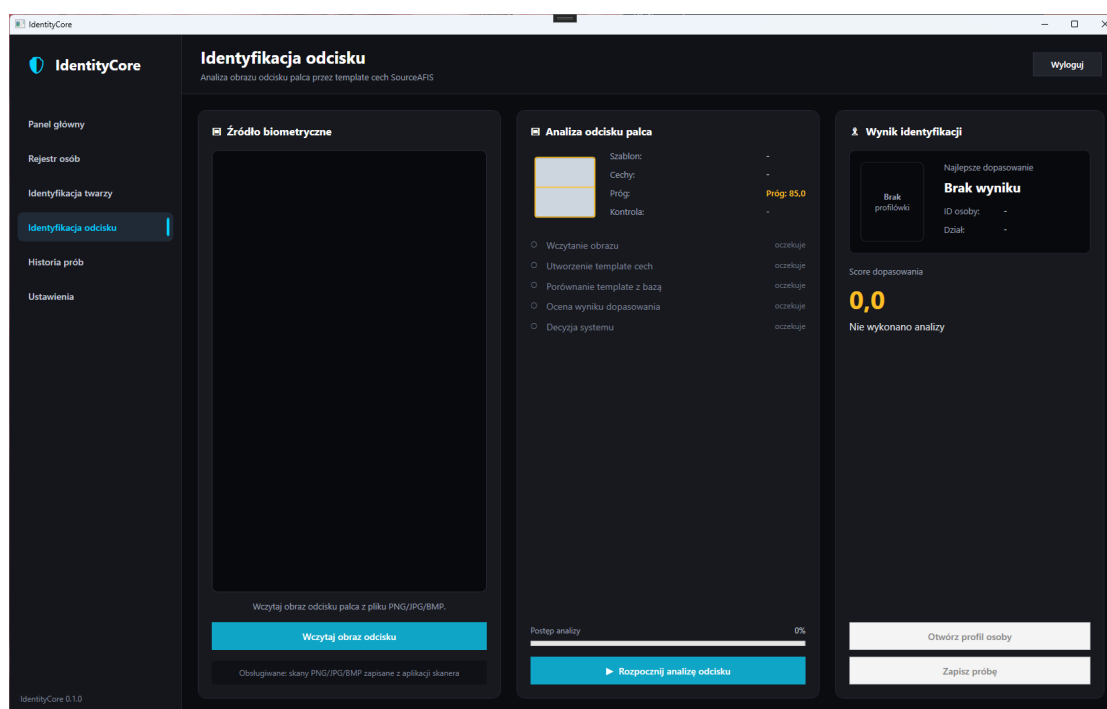


Rysunek 10.4. Widok identyfikacji twarzy przed uruchomieniem analizy

10.6. Widok identyfikacji odcisku palca

Widok identyfikacji odcisku palca ma podobny układ do modułu twarzy, ale jest dostosowany do pracy z obrazem odcisku. Lewy panel odpowiada za źródło biometryczne i umożliwia wczytanie obrazu odcisku palca z pliku. Widoczny jest duży obszar przeznaczony na podgląd próbki oraz przycisk **Wczytaj obraz odcisku**. Pod panelem znajduje się informacja o obsługiwanych formatach obrazów oraz o tym, że próbki mogą pochodzić z aplikacji skanera.

Środkowa część widoku pokazuje analizę odcisku palca. Operator widzi informacje o szablonie, cechach, progu oraz kontroli, a także kolejne etapy analizy: wczytanie obrazu, utworzenie template cech, porównanie template z bazą, ocenę wyniku dopasowania i decyzję systemu. Na dole znajduje się pasek postępu oraz przycisk rozpoczęcia analizy. Prawy panel prezentuje najlepsze dopasowanie, identyfikator osoby, dział, wynik punktowy oraz decyzję. W stanie początkowym widok pokazuje brak wyniku i informację, że analiza nie została jeszcze wykonana.

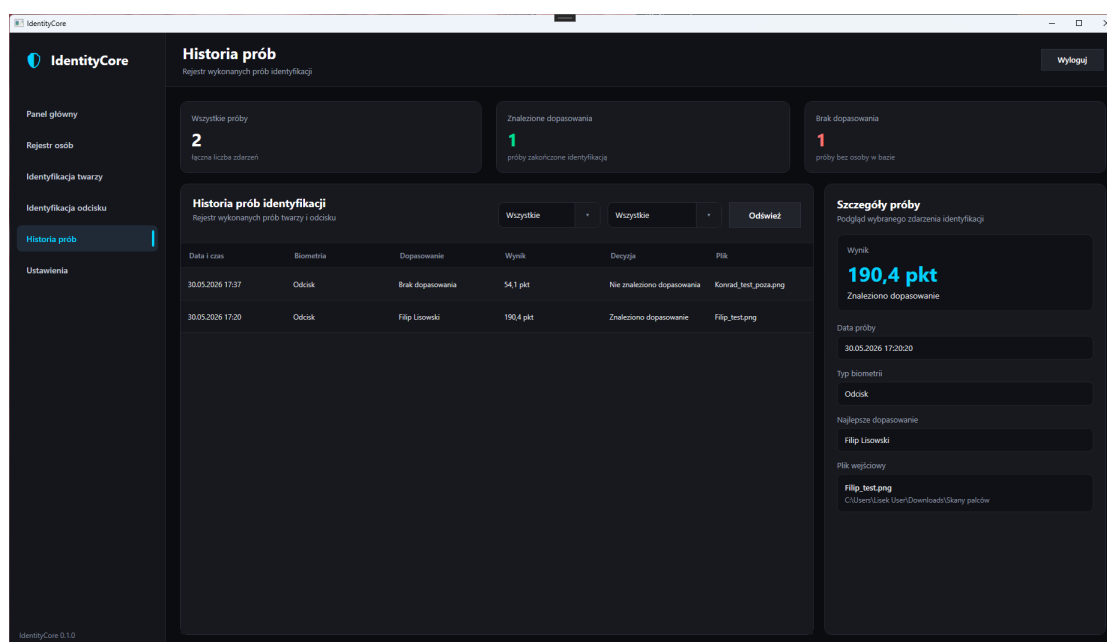


Rysunek 10.5. Widok identyfikacji odcisku palca przed uruchomieniem analizy

10.7. Historia prób identyfikacji

Widok historii prób identyfikacji służy do przeglądania zapisanych zdarzeń związanych z analizą twarzy i odcisku palca. W górnej części ekranu znajdują się liczniki przedstawiające łączną liczbę prób, liczbę znalezionych dopasowań oraz liczbę prób bez dopasowania. Niżej umieszczono tabelę historii, w której widoczne są takie informacje jak data i czas próby, typ biometrii, dopasowanie, wynik, decyzja oraz plik wejściowy. Nad tabelą znajdują się filtry oraz przycisk odświeżania danych.

Po prawej stronie widoku znajduje się panel szczegółów wybranej próby. Pokazuje on wynik, datę próby, typ biometrii, najlepsze dopasowanie oraz plik wejściowy. Taki układ pozwala operatorowi najpierw przeglądać zbiorczą listę prób, a następnie odczytać szczegóły konkretnego zdarzenia bez przechodzenia do innego modułu. Widok historii nie służy w tym rozdziale do oceny skuteczności systemu, ale pokazuje, w jaki sposób aplikacja prezentuje zapisane wyniki pracy modułów identyfikacji.

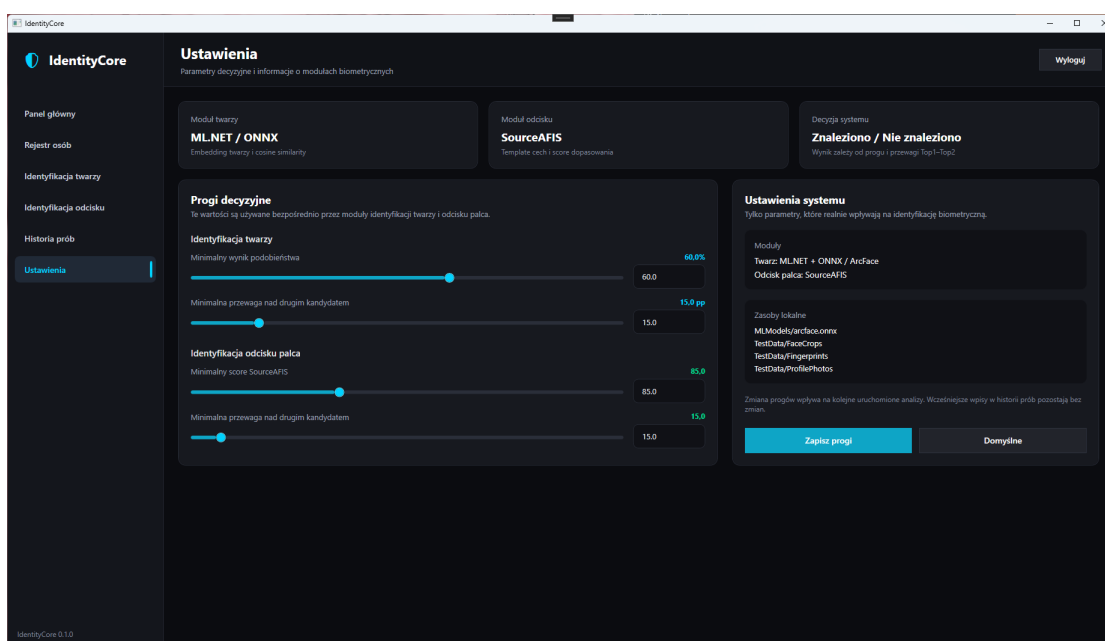


Rysunek 10.6. Widok historii prób identyfikacji

10.8. Ustawienia systemu

Widok ustawień prezentuje parametry decyzyjne oraz informacje o modułach biometrycznych. W górnej części ekranu znajdują się kafle opisujące moduł twarzy, moduł odcisku palca oraz sposób prezentacji decyzji systemu. W głównej części widoku umieszczono suwaki i pola wartości dla progów oraz marginesów: minimalnego wyniku podobieństwa twarzy, minimalnej przewagi nad drugim kandydatem, minimalnego wyniku SourceAFIS oraz minimalnej przewagi dla odcisku palca.

Prawa część widoku zawiera informacje o ustawieniach systemu, wykorzystywanych modułach oraz zasobach lokalnych. Widoczne są między innymi odniesienia do modelu `MLModels/arcface.onnx` oraz katalogów używanych przez aplikację. Na dole panelu znajdują się przyciski **Zapisz progi** i **Domyślne**. Pierwszy służy do zapisania aktualnie ustawionych wartości, a drugi do przywrócenia parametrów domyślnych. W tym rozdziale ustawienia są opisane wyłącznie jako element interfejsu, bez analizowania ich wpływu na wyniki identyfikacji.



Rysunek 10.7. Widok ustawień parametrów systemu

10.9. Komunikaty i informacje zwrotne dla operatora

Aplikacja przekazuje operatorowi informacje zwrotne bez konieczności otwierania osobnych okien dla każdego etapu pracy. W widokach identyfikacji widoczne są statusy etapów analizy, paski postępu, informacje o oczekiwaniu na wykonanie operacji oraz komunikaty o braku wyniku. Operator może więc kontrolować, czy dane wejściowe zostały wczytane, czy analiza została uruchomiona i czy system zakończył proces decyzję.

Wyniki pracy systemu są prezentowane w wydzielonych panelach. W zależności od modułu operator widzi najlepsze dopasowanie, identyfikator osoby, dział, wynik podobieństwa lub punktację, decyzję systemu oraz możliwość otwarcia profilu osoby lub zapisania próby. Historia prób pozwala natomiast wrócić do wcześniejszych zdarzeń i sprawdzić ich szczegóły. Wspólną cechą interfejsu jest wyraźne oddzielenie danych wejściowych, przebiegu analizy i rezultatu, co porządkuje sposób korzystania z aplikacji.

11. Wyniki, testy i analiza skuteczności

11.1. Opis przygotowanych testów

Testy aplikacji *IdentityCore* miały sprawdzić praktyczne działanie dwóch modułów biometrycznych: identyfikacji twarzy oraz identyfikacji odcisku palca. Zależało przede wszystkim na potwierdzeniu, czy system poprawnie zachowuje się w dwóch podstawowych sytuacjach: gdy próbka należy do osoby zapisanej w bazie oraz gdy próbka nie powinna zostać zaakceptowana jako dopasowanie.

Dla obu modalności przygotowano po dwa przypadki testowe. Pierwszy był przypadkiem pozytywnym, w którym oczekiwano znalezienia dopasowania do osoby zarejestrowanej w systemie. Drugi był przypadkiem negatywnym, w którym system powinien zwrócić brak dopasowania. W ten sposób możliwe było sprawdzenie nie tylko samego mechanizmu rozpoznawania, ale również działania progów decyzyjnych, sposobu prezentacji wyniku oraz zgodności zaprezentowanej decyzji z rzeczywistą sytuacją testową.

Próby nie miały charakteru szerokiej walidacji statystycznej. Ich celem było sprawdzenie, czy przygotowany prototyp działa zgodnie z założeniami projektowymi i czy oba moduły biometryczne potrafią zarówno wskazać poprawnego kandydata, jak i odrzucić próbkę niespełniającą warunków akceptacji.

11.2. Dane testowe

Do badań wykorzystano cztery próby: dwie dotyczące twarzy oraz dwie dotyczące odcisku palca. Zestaw został dobrany w sposób prosty, ale celowy: dla każdej modalności uwzględniono jedną próbkę, dla której system powinien znaleźć dopasowanie, oraz jedną próbkę, dla której powinien zwrócić brak dopasowania. Dane testowe zestawiono w tabeli 11.1.

Id	Biometria	Plik / źródło	Osoba oczekiwana	Charakter próbki
T1	Twarz	kamera	Konrad Fugiel	Próbka twarzy osoby zapisanej w bazie, dla której oczekiwano poprawnego dopasowania.
T2	Twarz	kamera	Brak dopasowania	Próbka twarzy osoby niewystępującej w bazie, przygotowana w celu sprawdzenia poprawnego odrzucenia.
O1	Odcisk palca	plik	Filip Lisowski	Próbka odcisku palca osoby zapisanej w bazie, dla której oczekiwano poprawnego dopasowania.
O2	Odcisk palca	plik	Brak dopasowania	Próbka odcisku palca, dla której system nie powinien znaleźć osoby w bazie.

Tabela 11.1. Zestaw danych testowych wykorzystanych podczas oceny działania systemu

Przygotowany zestaw jest niewielki, dlatego nie stanowi podstawy do wyznaczania miar statystycznych charakterystycznych dla pełnych badań biometrycznych. Pozwala jednak ocenić, czy system działa poprawnie w podstawowych, kontrolowanych scenariuszach pozytywnych i negatywnych.

11.3. Wyniki działania systemu

Wyniki działania systemu przedstawiono osobno dla identyfikacji twarzy i identyfikacji odcisku palca. Dla obu modułów zestawiono rezultat oczekiwany z rezultatem zwróconym przez aplikację, a następnie pokazano odpowiadające im zrzuty ekranu z wykonanych prób.

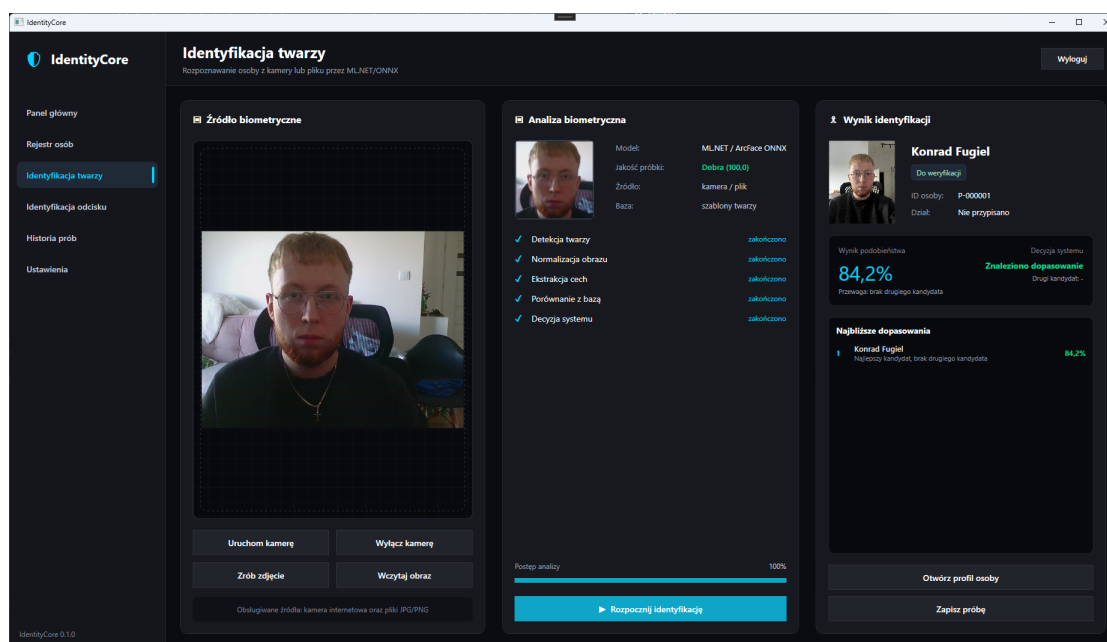
11.3.1. Wyniki identyfikacji twarzy

W module identyfikacji twarzy wykonano dwa testy. Pierwszy dotyczył osoby zapisanej w bazie i zakończył się poprawnym dopasowaniem. Drugi test dotyczył osoby niewystępującej w systemie i zakończył się brakiem dopasowania. Zestawienie wyników przedstawiono w tabeli 11.2.

Id	Oczekiwany wynik	Wynik systemu	Wynik dopasowania	Ocena
T1	Konrad Fugiel	Znaleziono dopasowanie: Konrad Fugiel	84,2%	Poprawne dopasowanie. Wynik przekroczył próg 60,0%.
T2	Brak dopasowania	Nie znaleziono osoby w bazie	41,4%	Brak dopasowania zgodny z oczekiwaniem. Wynik był poniżej progu 60,0%.

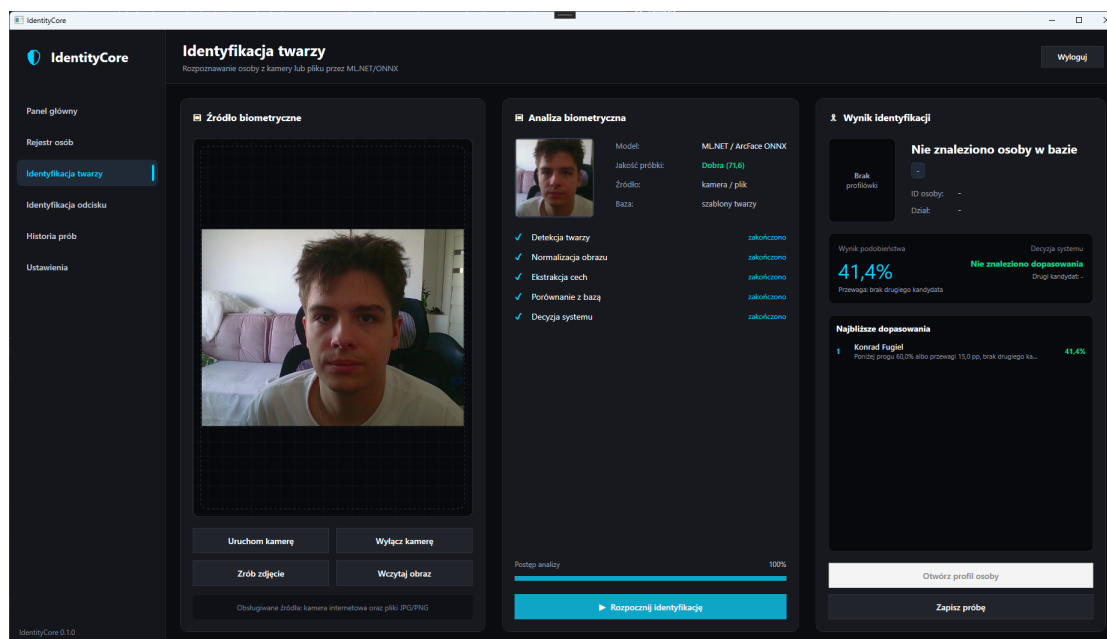
Tabela 11.2. Wyniki testów identyfikacji twarzy

Pierwsza próba twarzy zakończyła się poprawnym rozpoznaniem osoby zapisanej w bazie. System wskazał osobę *Konrad Fugiel*, a wynik podobieństwa osiągnął wartość 84,2%. W prawym panelu widoku wynikowego widoczna jest zarówno decyzja *Znaleziono dopasowanie*, jak i szczegóły najlepszego kandydata. Dla tej próby nie pojawił się drugi kandydat, dlatego decyzja została oparta na jednoznacznym najlepszym wyniku.



Rysunek 11.1. Wynik pozytywnej identyfikacji twarzy

Druga próba twarzy miała charakter negatywny. Do systemu wprowadzono twarz osoby niewystępującej w bazie, dlatego oczekiwano odrzucenia wyniku. System rzeczywiście nie zaakceptował dopasowania i zwrócił komunikat *Nie znaleziono osoby w bazie*. Najlepszy uzyskany wynik wyniósł 41,4%, co było wartością niższą od ustawionego progu 60,0%. W widoku wynikowym pokazano również najbliższe dopasowanie, jednak nie zostało ono zaakceptowane jako rozpoznanie.



Rysunek 11.2. Wynik negatywnej identyfikacji twarzy

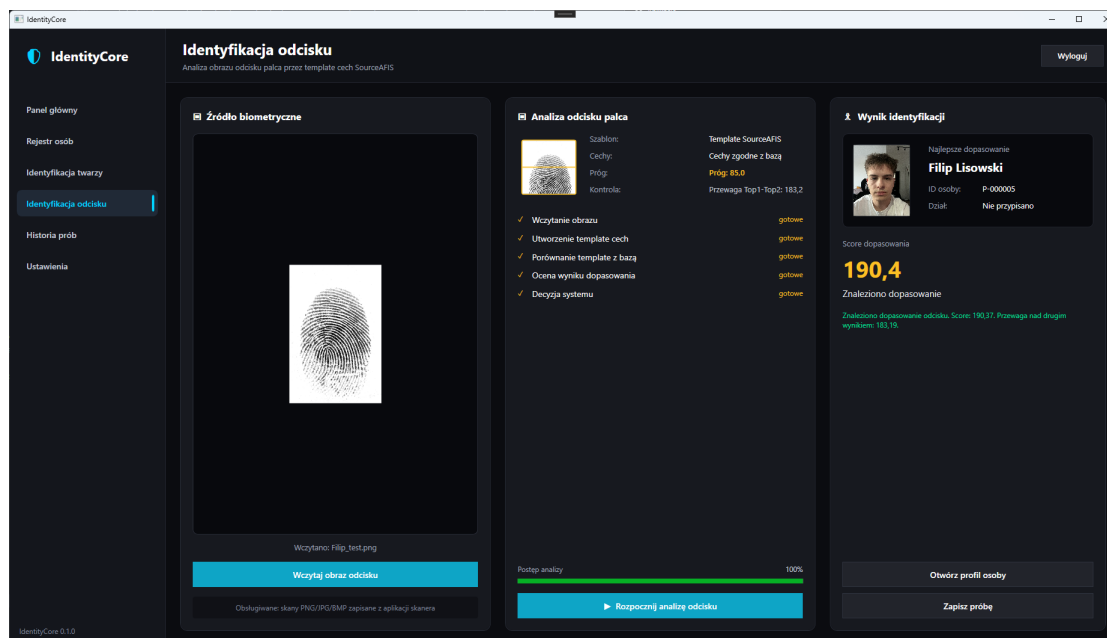
11.3.2. Wyniki identyfikacji odcisku palca

W module identyfikacji odcisku palca również przeprowadzono dwa testy: jeden pozytywny oraz jeden negatywny. W pierwszym przypadku system powinien wskazać osobę zapisaną w bazie, a w drugim zwrócić brak dopasowania. Wyniki zestawiono w tabeli 11.3.

Id	Oczekiwany wynik	Wynik systemu	Score dopasowania	Ocena
O1	Filip Lisowski	Znaleziono dopasowanie: Filip Lisowski	190,4	Poprawne dopasowanie. Wynik znacząco przekroczył próg 85,0.
O2	Brak dopasowania	Nie znaleziono osoby w bazie	9,6	Brak dopasowania zgodny z oczekiwaniem. Wynik był poniżej progu 85,0.

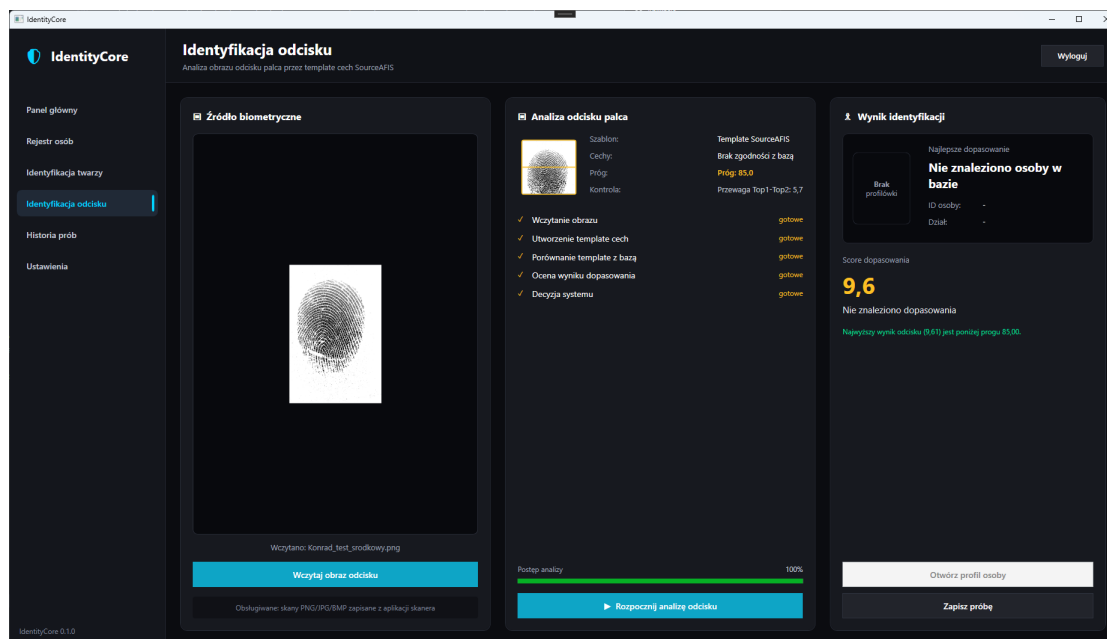
Tabela 11.3. Wyniki testów identyfikacji odcisku palca

Pozytywna próba odcisku palca została wykonana dla pliku graficznego. System wskazał osobę *Filip Lisowski*, a wynik dopasowania wyniósł 190,4, czyli wyraźnie przekroczył próg 85,0. Widok wynikowy pokazuje nie tylko decyzję *Znaleziono dopasowanie*, ale również identyfikator osoby oraz przewagę najlepszego wyniku nad kolejnym kandydatem.



Rysunek 11.3. Wynik pozytywnej identyfikacji odcisku palca

Negatywna próba odcisku palca została wykonana dla pliku *Konrad_test_srodkowy.png*. W tym przypadku system nie znalazł osoby w bazie. Najwyższy uzyskany wynik wyniósł 9,6, a więc był zdecydowanie niższy od progu 85,0. Na ekranie widoczna jest również przewaga Top1–Top2 równa 5,7, jednak już sam bardzo niski wynik dopasowania był wystarczający do odrzucenia próby.



Rysunek 11.4. Wynik negatywnej identyfikacji odcisku palca

11.4. Analiza poprawnych i błędnych rozpoznań

W przeprowadzonym zestawie testowym wystąpiły zarówno przypadki poprawnego dopasowania, jak i poprawnego braku dopasowania. Dla twarzy system prawidłowo rozpoznał osobę *Konrad Fugiel* przy wyniku 84,2% oraz prawidłowo odrzucił próbkę osoby niewystępującej w bazie przy wyniku 41,4%. Dla odcisku palca system poprawnie wskazał osobę *Filip Lisowski* przy wyniku 190,4 oraz poprawnie odrzucił próbkę negatywną z wynikiem 9,6.

W badanym zestawie nie wystąpił ani błędny brak dopasowania, ani błędne przypisanie próbki do niewłaściwej osoby. Wystąpiły natomiast dwa przypadki braku dopasowania i w obu sytuacjach był to rezultat zgodny z oczekiwaniem. Oznacza to, że w przygotowanych warunkach testowych system nie tylko znajdował poprawne dopasowania, ale potrafił również odrzucać próbki, które nie powinny zostać zaakceptowane.

Trzeba jednak podkreślić, że wnioski te dotyczą wyłącznie małego zestawu testowego. Wyniki mają charakter poglądowy i pokazują zachowanie prototypu w kilku kontrolowanych przypadkach, a nie pełną ocenę skuteczności systemu biometrycznego.

11.5. Wpływ parametrów na działanie systemu

W aplikacji zastosowano cztery główne parametry decyzyjne: dla twarzy próg podobieństwa 60,0% i margines 15,0 pp, a dla odcisku palca próg 85,0 i margines 15,0. W wykonanych próbach pozytywne przypadki przekroczyły wymagane progi, natomiast obie próby negatywne zostały odrzucone, ponieważ uzyskane wyniki były zbyt niskie.

W praktyce obniżenie progów mogłoby zwiększyć liczbę zaakceptowanych dopasowań, ale jednocześnie podniosłoby ryzyko błędnej akceptacji. Z kolei podwyższenie progów mogłoby poprawić ostrożność decyzji, ale kosztem częstszego odrzucania nawet poprawnych próbek. Dodatkową rolę pełni margines Top1–Top2, który pomaga odrzucać przypadki niejednoznaczne. Zmiana tych ustawień wpływa wyłącznie na kolejne próby i nie modyfikuje wcześniej zapisanych wyników w historii.

11.6. Wnioski z eksperymentów

Przeprowadzone testy potwierdziły, że aplikacja *IdentityCore* realizuje podstawowy cel demonstracyjny. Oba moduły biometryczne potrafiły poprawnie wskazać osobę zapisaną w bazie oraz zwrócić brak dopasowania w sytuacji, gdy próbka nie powinna zostać zaakceptowana.

Najważniejszym ograniczeniem badań jest mała liczba prób. Aby rzetelniej ocenić skuteczność systemu, należałoby przygotować większy i bardziej zróżnicowany zbiór danych, obejmujący więcej osób, więcej przypadków negatywnych oraz próbki o różnej jakości. Na obecnym etapie wyniki należy więc traktować jako potwierdzenie poprawnego działania prototypu, a nie jako pełną walidację skuteczności rozwiązania.

12. Analiza bezpieczeństwa systemu

12.1. Potencjalne zagrożenia

System *IdentityCore* jest lokalną aplikacją demonstracyjną, dlatego główne zagrożenia dotyczą przede wszystkim danych wejściowych, lokalnej bazy oraz konfiguracji parametrów decyzyjnych. Operator może wczytywać obrazy z plików, co oznacza, że jako dane wejściowe mogą zostać podane wcześniej przygotowane lub zmodyfikowane obrazy twarzy i odcisków palców. Ryzyko dotyczy więc nie tylko jakości próbek, ale również możliwości podstawienia danych nie pochodzących bezpośrednio od badanej osoby.

Drugim obszarem ryzyka jest lokalna baza danych, w której przechowywane są informacje o osobach, szablony biometryczne, ustawienia i historia prób. Podmiana, uszkodzenie lub nieuprawniona modyfikacja tych danych mogłaby wpłynąć na działanie systemu. Znaczenie mają również progi decyzyjne: zbyt niskie wartości mogą zwiększać ryzyko błędnej akceptacji, a zbyt wysokie mogą powodować częstsze odrzucanie poprawnych próbek.

12.2. Ryzyko fałszowania danych biometrycznych

Istotnym ograniczeniem prototypu jest brak pełnego mechanizmu wykrywania żywotności próbki, czyli *liveness detection*. Oznacza to, że system może być podatny na próby użycia zdjęcia twarzy zamiast żywej osoby, obrazu twarzy wyświetlonego na ekranie albo wcześniej przygotowanego pliku graficznego. Podobne ryzyko dotyczy odcisków palców, gdzie do aplikacji można wprowadzić skan lub kopię odcisku zapisaną jako obraz.

W obecnej wersji aplikacja analizuje dostarczony obraz i podejmuje decyzję na podstawie wyniku dopasowania, progu oraz marginesu. Nie sprawdza jednak w zaawansowany sposób, czy próbka pochodzi od osoby obecnej fizycznie przy stanowisku.

12.3. Błędy identyfikacji

W systemach biometrycznych mogą wystąpić dwa podstawowe typy błędów: fałszywa akceptacja oraz fałszywe odrzucenie. Fałszywa akceptacja oznacza uznanie próbki za zgodną z osobą z bazy mimo braku rzeczywistej zgodności. Fałszywe odrzucenie oznacza brak dopasowania mimo tego, że próbka pochodzi od osoby zapisanej w systemie.

Na decyzję wpływa jakość danych wejściowych. W przypadku twarzy znaczenie mają oświetlenie, ostrość, ustawienie głowy i widoczność twarzy. W przypadku odcisku palca istotne są czytelność linii papilarnych, kompletność próbki, kontrast i ewentualne rozmazanie. Dodatkowym zabezpieczeniem jest margines między najlepszym i drugim kandydatem, który ogranicza akceptację przypadków niejednoznacznych.

12.4. Ocena odporności systemu

Aplikacja posiada podstawowe mechanizmy ograniczające ryzyko błędnej decyzji. Należą do nich progi decyzyjne, margines Top1–Top2, odrzucanie wyników poniżej progu, komunikaty o nieudanej analizie oraz historia prób identyfikacji.

Jednocześnie odporność systemu jest ograniczona. Prototyp nie posiada pełnego wykrywania żywotności próbki, nie zawiera rozbudowanego modelu ról użytkowników i nie powinien być traktowany jako system produkcyjnie bezpieczny.

12.5. Możliwe usprawnienia bezpieczeństwa

Najważniejszym kierunkiem rozwoju byłoby dodanie mechanizmów *liveness detection* dla identyfikacji twarzy, aby ograniczyć ryzyko użycia zdjęcia lub obrazu wyświetlonego na ekranie. W module odcisku palca warto rozważyć bezpośrednią integrację ze skanerem, zamiast opierania procesu wyłącznie na wczytywaniu plików graficznych.

Kolejne usprawnienia powinny dotyczyć ochrony danych i dostępu do aplikacji. W przyszłości warto dodać hashowanie i solenie haseł operatora, blokadę po wielu błędnych próbach logowania, szyfrowanie szablonów biometrycznych oraz dokładniejsze rejestrowanie zdarzeń administracyjnych. Dodatkowo system powinien zostać sprawdzony na większym zbiorze danych, z uwzględnieniem testów fałszywej akceptacji i fałszywego odrzucenia.

13. Harmonogram realizacji projektu

13.1. Etapy realizacji

Prace nad projektem *IdentityCore* rozpoczęto na początku maja 2026 roku, a zakończono w drugim tygodniu czerwca 2026 roku. Projekt był realizowany etapowo, jednak poszczególne części częściowo nakładały się na siebie. Wynikało to z charakteru aplikacji, w której warstwa interfejsu, baza danych, logika biometryczna i testy musiały być rozwijane równolegle.

Pierwszym etapem było określenie zakresu projektu oraz wybór głównych funkcji systemu. Ustalono, że aplikacja będzie lokalnym systemem identyfikacji osób, wykorzystującym dwa typy danych biometrycznych: obraz twarzy oraz obraz odcisku palca. Na tym etapie doprecyzowano również, że projekt będzie aplikacją desktopową z bazą danych SQL Server.

Kolejne etapy obejmowały przygotowanie struktury aplikacji, modelu danych oraz widoków WPF. Następnie rozwijano rejestr osób, obsługę profilu osoby, zapisywanie zdjęcia profilowego oraz powiązanie osoby z próbkami biometrycznymi. Po przygotowaniu podstaw aplikacji rozpoczęto implementację modułów identyfikacji twarzy i odcisku palca.

W końcowej części prac skupiono się na dopracowaniu interfejsu, historii prób, ustawieniach systemowych, testach oraz dokumentacji. Ostatni etap obejmował poprawki, uporządkowanie plików SQL, przygotowanie zrzutów ekranu i opis działania systemu w pracy projektowej.

13.2. Harmonogram prac

Harmonogram prac został przygotowany w ujęciu tygodniowym. Dаты mają charakter porządkujący i odzwierciedlają główne etapy pracy nad aplikacją.

Etap	Zakres prac	Termin	Rezultat
1	Analiza założeń i zakresu projektu	04.05–08.05.2026	Ustalenie zakresu funkcji oraz głównych modułów aplikacji.
2	Przygotowanie struktury projektu i bazy danych	09.05–15.05.2026	Utworzenie podstaw aplikacji, struktury katalogów, modelu danych i skryptów SQL.
3	Rejestr osób i profil osoby	16.05–22.05.2026	Przygotowanie obsługi osób, edycji profilu, zdjęcia profilowego oraz próbek biometrycznych.
4	Moduł identyfikacji twarzy	23.05–29.05.2026	Dodanie obsługi obrazu twarzy, modelu <code>arcface.onnx</code> , porównywania embeddingów i decyzji systemu.
5	Moduł identyfikacji odcisku palca	27.05–02.06.2026	Dodanie obsługi obrazów odcisków, szablonów SourceAFIS, porównywania wzorców i decyzji systemu.

Etap	Zakres prac	Termin	Rezultat
6	Historia, ustawienia i panel główny	03.06–07.06.2026	Przygotowanie historii prób, parametrów decyzyjnych, podsumowań systemu i szybkich przejść.
7	Testy, poprawki i finalizacja aplikacji	08.06–11.06.2026	Sprawdzenie działania modułów, korekty błędów, przygotowanie danych testowych i zrzutów ekranu.
8	Dokumentacja projektu	10.06–14.06.2026	Przygotowanie opisu projektu, rozdziałów dokumentacji, wyników testów i materiałów końcowych.

Tabela 13.1. Harmonogram realizacji projektu IdentityCore

13.3. Podział pracy

Projekt był realizowany zespołowo. Prace nie zostały sztywno podzielone na całkowicie oddzielne części wykonywane niezależnie przez poszczególne osoby. Zespół pracował wspólnie nad najważniejszymi decyzjami projektowymi, zakresem funkcji, wyglądem aplikacji, testami oraz dokumentacją.

Wspólny charakter pracy był szczególnie istotny przy modułach biometrycznych, ponieważ identyfikacja twarzy, identyfikacja odcisku palca, rejestr osób i historia prób są ze sobą powiązane. Zmiana w jednym module często wymagała sprawdzenia działania pozostałych elementów aplikacji. Z tego powodu wiele zadań było konsultowanych i testowanych wspólnie.

W praktyce praca zespołu obejmowała wspólne ustalanie wymagań, projektowanie struktury aplikacji, przygotowywanie widoków, implementację funkcji, testowanie działania systemu oraz opracowanie dokumentacji. Taki sposób pracy pozwolił zachować spójność projektu i uniknąć sytuacji, w której poszczególne moduły działałyby osobno, ale nie tworzyłyby jednego kompletnego systemu.

14. Instrukcja uruchomienia projektu

14.1. Wymagania systemowe

Do uruchomienia aplikacji *IdentityCore* wymagane jest lokalne stanowisko z systemem Windows oraz środowiskiem umożliwiającym kompilację aplikacji desktopowej WPF/.NET. Projekt był przygotowywany i uruchamiany w środowisku Visual Studio 2026 w wersji 18.5.2, z projektem docelowym ustawionym na `net10.0-windows`.

Aplikacja korzysta również z lokalnej bazy danych SQL Server oraz modelu `arcface.onnx`, który jest przechowywany w projekcie jako plik zarządzany przez Git LFS. Z tego powodu samo pobranie repozytorium nie jest wystarczające, jeżeli pliki LFS nie zostaną pobrane poprawnie.

Element	Wymaganie
System operacyjny	Windows z obsługą aplikacji desktopowych .NET/WPF.
Środowisko programistyczne	Visual Studio 2026, wersja 18.5.2.
Platforma .NET	.NET 10, projekt docelowy <code>net10.0-windows</code> .
Baza danych	SQL Server oraz SQL Server Management Studio.
Repozytorium	Git oraz Git LFS do pobrania kodu i dużych plików projektu.
Model twarzy	Plik <code>IdentityCore.App/MLModels/arcface.onnx</code> .

Tabela 14.1. Podstawowe wymagania potrzebne do uruchomienia projektu

14.2. Konfiguracja środowiska

Pierwszym krokiem jest pobranie projektu z repozytorium. Ponieważ projekt wykorzystuje plik modelu `arcface.onnx`, należy upewnić się, że na stanowisku zainstalowany jest Git LFS. Po sklonowaniu repozytorium trzeba pobrać pliki zarządzane przez LFS.

Listing 14.1. Przykładowe pobranie projektu i plików Git LFS

```
1 git clone https://github.com/KonradF20/IdentityCore.git
2 cd IdentityCore
3 git lfs pull
```

Po wykonaniu tych poleceń należy sprawdzić, czy w projekcie znajduje się plik `IdentityCore.App/MLModels/arcface.onnx`. Brak tego pliku uniemożliwi poprawne działanie modułu identyfikacji twarzy.

Następnie projekt należy otworzyć w Visual Studio. Po otwarciu rozwiązania środowisko powinno przywrócić pakiety NuGet automatycznie. Jeżeli tak się nie stanie, należy użyć opcji przywrócenia pakietów z poziomu Visual Studio albo wykonać polecenie `dotnet restore`. W ustawieniach projektu należy również sprawdzić, czy aplikacja jest kompilowana dla platformy `net10.0-windows`.

14.3. Konfiguracja bazy danych

Aplikacja korzysta z lokalnej bazy SQL Server. Przed uruchomieniem programu należy upewnić się, że usługa SQL Server działa, a następnie otworzyć SQL Server Management Studio i połączyć się z właściwą instancją serwera.

Do przygotowania bazy od podstaw służy skrypt `create_database.sql`. Skrypt tworzy bazę `IdentityCoreDb`, tabele wymagane przez aplikację, relacje, indeksy oraz dane startowe potrzebne do pracy systemu. Skrypt należy uruchomić w SQL Server Management Studio.

Po wykonaniu skryptu trzeba sprawdzić, czy aplikacja łączy się z bazą danych. Jeżeli połączenie nie działa, należy dostosować `connection string` zapisany w projekcie do lokalnej konfiguracji SQL Server, w szczególności nazwę instancji serwera oraz nazwę bazy danych.

Po stronie osoby uruchamiającej projekt konieczne jest sprawdzenie, czy `connection string` wskazuje na lokalną instancję SQL Server oraz bazę `IdentityCoreDb`. W przypadku innej nazwy instancji SQL Server należy dostosować dane połączenia w projekcie do konfiguracji danego stanowiska.

14.4. Uruchomienie aplikacji

Zalecany sposób uruchomienia projektu jest użycie Visual Studio. Po otwarciu rozwiązania należy ustawić projekt `IdentityCore.App` jako projekt startowy, a następnie skompilować rozwiązanie. Jeżeli kompilacja zakończy się poprawnie, aplikację można uruchomić przyciskiem startowym w Visual Studio.

Po uruchomieniu powinien pojawić się ekran logowania. Po zalogowaniu operator może sprawdzić podstawowe moduły aplikacji: panel główny, rejestr osób, identyfikację twarzy, identyfikację odcisku palca, historię prób oraz ustawienia systemowe.

Projekt można również uruchomić z poziomu terminala, o ile stanowisko ma poprawnie skonfigurowane środowisko .NET i znajduje się w katalogu projektu.

Listing 14.2. Przykładowe uruchomienie aplikacji z poziomu katalogu projektu

```
1 dotnet restore
2 dotnet build
3 dotnet run --project IdentityCore.App
```

Jeżeli struktura katalogów w lokalnej kopii różni się od powyższego przykładu, ścieżkę po parametrze `--project` należy dopasować do lokalizacji pliku `IdentityCore.App.csproj`.

14.5. Dane testowe i konta testowe

Do paczki projektu nie dołączono pełnego zestawu danych biometrycznych używanego podczas testów. Wynika to z charakteru danych przetwarzanych przez aplikację: obrazy twarzy oraz odcisków palców mogą być traktowane jako dane wrażliwe i nie powinny być udostępniane bez wyraźnej potrzeby. Testy opisane w pracy zostały wykonane na lokalnie przygotowanych próbkach zespołu.

W celu ponownego sprawdzenia działania aplikacji osoba uruchamiająca projekt powinna przygotować własne dane testowe. Dla modułu twarzy można wykorzystać obraz z kamery albo plik graficzny przedstawiający twarz. Dla modułu odcisku palca należy przygotować obraz odcisku zapisany w formacie obsługiwany przez aplikację, na przykład PNG, JPG lub BMP. Dane powinny zostać podzielone na próbki referencyjne dodawane do profilu osoby oraz próbki wejściowe wykorzystywane później do identyfikacji.

Zalecany sposób przygotowania prostego zestawu testowego jest następujący: najpierw należy utworzyć w rejestrze osobę testową, następnie zarejestrować dla niej próbkę twarzy albo odcisku palca, a dopiero później uruchomić identyfikację na drugiej próbce tej samej osoby. Do sprawdzenia przypadku negatywnego

można użyć próbki innej osoby, która nie została dodana do rejestru. Dzięki temu możliwe jest sprawdzenie zarówno dopasowania, jak i braku dopasowania.

Pliki robocze generowane lub kopiowane przez aplikację są zapisywane lokalnie w katalogu uruchomieniowym programu, czyli w strukturze tworzonej podczas kompilacji pod katalogiem `bin`. Dokładna ścieżka może zależeć od trybu uruchomienia, konfiguracji kompilacji oraz ustawień środowiska Visual Studio. Z tego powodu po pierwszym uruchomieniu aplikacji warto sprawdzić katalog wynikowy projektu `IdentityCore.App`, w szczególności podkatalogi utworzone obok pliku wykonywalnego.

Do zalogowania w środowisku demonstracyjnym wykorzystywane jest konto testowe:

Listing 14.3. Dane konta testowego operatora

```
1 login: admin
2 haslo: admin
```

Po zalogowaniu można sprawdzić działanie podstawowych modułów: przejrzeć rejestr osób, dodać osobę testową, zarejestrować próbki biometryczne, uruchomić identyfikację twarzy lub odcisku palca, zapisać próbę do historii oraz sprawdzić wartości progów w ustawieniach systemu.

15. Podsumowanie i wnioski końcowe

15.1. Ocena realizacji założeń

Celem projektu było przygotowanie lokalnej aplikacji desktopowej umożliwiającej identyfikację osób na podstawie danych biometrycznych twarzy oraz odcisku palca. Założenie to zostało zrealizowane w postaci aplikacji *IdentityCore*, która łączy rejestr osób, obsługę próbek biometrycznych, proces identyfikacji, historię prób oraz ustawienia parametrów decyzyjnych.

W projekcie udało się przygotować spójny system, w którym osoba może zostać dodana do rejestru, uzupełniona o dane profilowe i powiązana z materiałem biometrycznym. Aplikacja pozwala następnie uruchomić identyfikację twarzy lub odcisku palca, zaprezentować wynik operatorowi oraz zapisać próbę w historii. Zrealizowano również panel główny, ekran logowania, widok ustawień oraz widok historii, dzięki czemu aplikacja nie ogranicza się wyłącznie do pojedynczego algorytmu, ale tworzy pełniejszy prototyp systemu identyfikacji.

Założenia projektowe zostały więc spełnione na poziomie demonstracyjnym. System pokazuje kompletny przepływ pracy: od rejestracji osoby, przez przygotowanie danych biometrycznych, po wykonanie próby identyfikacji i zapis wyniku.

15.2. Napotkane problemy

Największe trudności dotyczyły integracji części biometrycznej z aplikacją desktopową i bazą danych. W przypadku identyfikacji twarzy istotnym problemem było poprawne przygotowanie obrazu przed przekazaniem go do modelu. Należało zadbać o wykrycie twarzy, przygotowanie odpowiedniego wycinka obrazu, wygenerowanie reprezentacji twarzy oraz porównanie jej z zapisanymi wzorcami.

W przypadku odcisków palców problemem było przygotowanie takiego sposobu pracy, który pozwalałby analizować obrazy zapisane w plikach. Ponieważ projekt nie opierał się na pełnej integracji ze skanem linii papilarnych, konieczne było przyjęcie rozwiązania polegającego na wczytywaniu obrazów odcisków. Pozwoliło to zrealizować moduł identyfikacji, ale jednocześnie ujawniło ograniczenia prototypu.

15.3. Ocena skuteczności rozwiązania

Przeprowadzone testy pokazały, że aplikacja działa zgodnie z założeniami w podstawowych scenariuszach. System potrafił wskazać osobę zapisaną w bazie oraz odrzucić próbkę, której wynik znajdował się poniżej przyjętego progu decyzyjnego. Dotyczyło to zarówno modułu identyfikacji twarzy, jak i modułu identyfikacji odcisku palca.

Wyniki należy jednak interpretować ostrożnie. Testy miały charakter projektowy i zostały wykonane na niewielkim zbiorze danych. Nie można na ich podstawie traktować systemu jako rozwiązania produkcyjnego ani wyciągać pełnych wniosków statystycznych dotyczących skuteczności biometrycznej. Aplikacja spełnia natomiast rolę prototypu pokazującego praktyczne działanie procesu identyfikacji.

Istotne znaczenie dla skuteczności mają jakość danych wejściowych, liczba zapisanych wzorców, wartości progów decyzyjnych oraz margines między najlepszymi kandydatami. W obecnej wersji system sprawdza te warunki i potrafi odrzucić próbkę, jeżeli wynik jest zbyt niski lub nie spełnia przyjętych kryteriów.

15.4. Możliwości zastosowania systemu

IdentityCore może służyć jako demonstracyjny system identyfikacji osób w środowisku lokalnym. Projekt dobrze pokazuje, jak można połączyć rejestr osób, dane biometryczne, bazę danych oraz aplikację desktopową w jednym narzędziu.

Aplikacja może znaleźć zastosowanie w sytuacjach, w których potrzebne jest pokazanie zasady działania identyfikacji biometrycznej bez wdrażania pełnego systemu produkcyjnego. Przykładem może być prezentacja działania rozpoznawania twarzy, porównywania odcisków palców, zapisu historii prób oraz wpływu parametrów decyzyjnych na wynik.

Należy podkreślić, że system nie posiada pełnych mechanizmów bezpieczeństwa wymaganych w rozwiązaniach produkcyjnych, takich jak wykrywanie żywotności próbki, zaawansowana ochrona szablonów biometrycznych czy rozbudowane zarządzanie uprawnieniami.

15.5. Kierunki dalszego rozwoju

Dalszy rozwój systemu powinien obejmować przede wszystkim zwiększenie bezpieczeństwa i odporności modułów biometrycznych. W przypadku identyfikacji twarzy ważnym kierunkiem byłoby dodanie mechanizmu wykrywania żywotności próbki, aby ograniczyć ryzyko użycia zdjęcia lub obrazu wyświetlonego na ekranie. W module odcisku palca warto rozważyć bezpośrednią integrację ze skanerem, zamiast opierania procesu wyłącznie na wczytywaniu plików graficznych.

Kolejnym kierunkiem rozwoju jest rozszerzenie testów na większym i bardziej zróżnicowanym zbiorze danych. Pozwoliłoby to dokładniej ocenić zachowanie progów decyzyjnych, liczbę błędnych akceptacji oraz liczbę błędnych odrzuceń. W przyszłości można również rozbudować ochronę danych, między innymi przez szyfrowanie szablonów biometrycznych, lepsze zabezpieczenie logowania operatora oraz dokładniejsze rejestrowanie zdarzeń administracyjnych.

Bibliografia

- [1] Apple. Face id security guide. https://www.apple.com/business-docs/FaceID_Security_Guide.pdf, 2026.
- [2] Jiankang Deng, Jia Guo, Niannan Xue, and Stefanos Zafeiriou. Arcface: Additive angular margin loss for deep face recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2019.
- [3] HID Global. Hid signo biometric reader 25b. <https://www.hidglobal.com/products/25b>, 2026.
- [4] Microsoft. Configure windows hello. <https://support.microsoft.com/en-us/windows/configure-windows-hello-dae28983-8242-bb2a-d3d1-87c9d265a5f0>, 2026.
- [5] Neurotechnology. Megamatcher sdk. <https://www.neurotechnology.com/megamatcher.html>, 2026.
- [6] ONNX. Open neural network exchange. <https://onnx.ai/>, 2026.
- [7] Robert Vazan. Sourceafis fingerprint matcher. <https://sourceafis.machinezoo.com/>, 2026.

Spis rysunków

4.1	Ogólny pipeline działania systemu biometrycznego IdentityCore	20
4.2	Proces rejestracji osoby w systemie <i>IdentityCore</i>	21
4.3	Schemat blokowy działania systemu <i>IdentityCore</i>	22
8.1	Diagram ERD bazy danych systemu IdentityCore	35
10.1	Ekran logowania do aplikacji IdentityCore	50
10.2	Panel główny aplikacji po zalogowaniu	51
10.3	Rejestr osób wraz z panelem profilu wybranej osoby	52
10.4	Widok identyfikacji twarzy przed uruchomieniem analizy	53
10.5	Widok identyfikacji odcisku palca przed uruchomieniem analizy	54
10.6	Widok historii prób identyfikacji	55
10.7	Widok ustawień parametrów systemu	56
11.1	Wynik pozytywnej identyfikacji twarzy	58
11.2	Wynik negatywnej identyfikacji twarzy	59
11.3	Wynik pozytywnej identyfikacji odcisku palca	60
11.4	Wynik negatywnej identyfikacji odcisku palca	60

Spis tabel

3.1	Wymagania funkcjonalne systemu	17
3.2	Wymagania нефункционалне systemu	17
3.3	Wymagania dotyczące bezpieczeństwa	18
6.1	Główne warstwy i moduły aplikacji	27
7.1	Najważniejsze pakiety NuGet wykorzystane w projekcie	31
7.2	Podsumowanie technologii wykorzystanych w projekcie	32
8.1	Główne tabele bazy danych systemu IdentityCore	34
9.1	Najważniejsze moduły implementacyjne aplikacji IdentityCore	38
11.1	Zestaw danych testowych wykorzystanych podczas oceny działania systemu	57
11.2	Wyniki testów identyfikacji twarzy	58
11.3	Wyniki testów identyfikacji odcisku palca	59
13.1	Harmonogram realizacji projektu IdentityCore	65
14.1	Podstawowe wymagania potrzebne do uruchomienia projektu	66

Spis listingów

9.1	Przygotowanie obrazu twarzy przed analizą	39
9.2	Użycie modelu arcface.onnx do wygenerowania embeddingu twarzy	40
9.3	Wybór kandydatów podczas identyfikacji twarzy	41
9.4	Obliczanie podobieństwa cosinusowego dla embeddingów twarzy	42
9.5	Utworzenie szablonu odcisku palca na podstawie obrazu wejściowego	43
9.6	Porównywanie próbki odcisku palca z zapisanymi szablonami	45
9.7	Decyzja identyfikacyjna na podstawie progu i marginesu dopasowania	45
9.8	Zapis próby identyfikacji do historii	46
9.9	Sprawdzenie danych logowania operatora	47
14.1	Przykładowe pobranie projektu i plików Git LFS	66
14.2	Przykładowe uruchomienie aplikacji z poziomu katalogu projektu	67
14.3	Dane konta testowego operatora	68

Załącznik nr 2 do Zarządzenia nr 228/2021 Rektora Uniwersytetu Rzeszowskiego z dnia 1 grudnia 2021 roku w sprawie ustalenia procedury antyplagiatowej w Uniwersytecie Rzeszowskim

OŚWIADCZENIE STUDENTA O SAMODZIELNOŚCI PRACY

..... Filip Lisowski, Konrad Fugiel

Imię (imiona) i nazwisko studenta

Wydział Nauk Ścisłych i Technicznych

..... Informatyka

Nazwa kierunku

..... 134942, 134909

Numer albumu

1. Oświadczam, że moja praca projektowa pt.: Dokumentacja projektowa - Wewnętrzny system identyfikacji osób
 - 1) została przygotowana przeze mnie samodzielnie*,
 - 2) nie narusza praw autorskich w rozumieniu ustawy z dnia 4 lutego 1994 roku o prawie autorskim i prawach pokrewnych (t.j. Dz.U. z 2021 r., poz. 1062) oraz dóbr osobistych chronionych prawem cywilnym,
 - 3) nie zawiera danych i informacji, które uzyskałem/am w sposób niedozwolony,
 - 4) nie była podstawą otrzymania oceny z innego przedmiotu na uczelni wyższej ani mnie, ani innej osobie.
2. Jednocześnie wyrażam zgodę/nie wyrażam zgody** na udostępnienie mojej pracy projektowej do celów naukowo-badawczych z poszanowaniem przepisów ustawy o prawie autorskim i prawach pokrewnych.

(miejscowość, data)

(czytelny podpis studenta)

* Uwzględniając merytoryczny wkład prowadzącego przedmiot

** – niepotrzebne skreślić